

Generative Plagiarism Detection System

Junior Project – Completed the requirements for obtaining a bachelor's
degree in Artificial Intelligence Engineering

Prepared by:

Farah Alkayyem Tala Alkhateeb

Supervised by:

Dr. Riad Sonbol
Eng. Raghad Alhossny

Academic Year: 2025 – 2026

Abstract

The increasing accessibility of large language models has given rise to a new form of academic misconduct known as generative plagiarism, in which AI-generated text is submitted as original work despite being semantically derived from existing academic sources. Traditional plagiarism detection tools are largely ineffective against this form of dishonesty, as the generated text shares no verbatim overlap with the underlying sources.

This report presents a Generative Plagiarism Detection system — a full-stack web platform designed to help both students and instructors identify potential AI-generated plagiarism in academic submissions. The system provides a structured environment for uploading documents, managing submissions, and receiving detailed plagiarism analysis reports. The actual detection logic is handled by a dedicated AI microservice, keeping the main application focused on user experience and workflow management.

Table of Contents

Abstract	2
Table of Contents	3
Chapter 1: Introduction	5
1.1 Introduction.....	5
1.2 Problem Statement.....	5
1.3 Project Objectives	5
1.4 Proposed System.....	6
1.5 Report Organization.....	6
1.6 Summary	7
Chapter 2: Fundamental Concepts and Literature Review	8
2.1 Introduction.....	8
2.2 Fundamental Concepts.....	8
2.3 Literature Review of Existing Systems.....	9
Turnitin	9
iThenticate.....	10
Copyleaks.....	11
Originality.AI.....	12
2.4 Literature Review of AI Service	13
2.5 Summary	17
Chapter 3: Project Management.....	18
3.1 Introduction.....	18
3.2 Project Charter	18
3.3 Roles and Responsibilities	19
3.4 Statement of Work (SOW).....	19
3.4.1 Project Description and Objectives.....	19
3.4.2 Project Scope	19
3.4.3 Project Goals.....	20
3.4.4 Deliverables	20
3.4.5 Assumptions.....	21
3.4.6 Project Resources.....	21
3.4.7 Schedule.....	21
3.4.8 Risk Management	22

3.5 Summary	22
Chapter 4: Requirements and System Analysis	24
4.1 Introduction.....	24
4.2 Project Timeline.....	24
Software Requirement Specification Document (SRS).....	25
1. Introduction.....	25
2. Overall Description.....	27
3. System Features	30
4. System Requirements	35
5. Requirements Modeling.....	38

Chapter 1: Introduction

1.1 Introduction

This chapter introduces the core elements of the **Generative Plagiarism Detection** system, laying the foundation for understanding the challenges of academic integrity in the age of AI, the project goals, the proposed microservice-based architecture, the organization of the report, and a brief summary of key points.

1.2 Problem Statement

Academic institutions currently lack dedicated software platforms for managing AI-generated plagiarism detection workflows at scale. Existing tools were designed for surface-level text matching and are not equipped to identify documents that are semantically derived from sources without sharing verbatim text. Beyond the detection algorithm itself, there is a need for a structured platform that handles the full workflow: document upload and management, asynchronous analysis processing, plan-specific access to system features, and clear presentation of analysis results.

The specific problems this project addresses are:

- There is no standardized workflow tool for submitting documents for AI plagiarism analysis and reviewing structured results.
- Document analysis can be time-consuming and must not block the user interface or time out web requests.
- The AI detection logic is complex and evolving; it must be decoupled from the main application so it can be developed and updated independently.
- The system must serve multiple user types — students and instructors — with appropriately different permissions and views.

1.3 Project Objectives

The objectives of this project are:

1. Design and develop a full-stack web application that supports document upload, submission management, and plagiarism analysis for academic use.
2. Implement asynchronous processing using Celery so that document analysis does not block the user interface.

3. Integrate the web platform with a separately developed AI microservice via a well-defined HTTP API, maintaining a clean separation of concerns.
4. Develop an optimized AI processing pipeline to handle model inference efficiently, ensuring relatively rapid generation of analysis results.
5. Implement plan-based access control to deliver specific features and workflows according to the user's subscription level.
6. Present analysis results in a clear, structured format that allows users to understand the degree and origin of detected plagiarism.
7. Follow sound software engineering practices throughout: modular design, documented APIs, version-controlled development, and systematic testing.

1.4 Proposed System

The proposed system consists of two integrated components:

Web Application Platform — the primary deliverable of this project. Built with Django (backend), React (frontend), SQL-Server (database), and Celery with RabbitMQ (task queue). It handles all user-facing functionality: authentication and role management, document and source upload, submission workflows, asynchronous analysis job queuing, result storage, and result display.

AI Analysis Microservice — a FastAPI service responsible for the plagiarism detection logic. The service processes suspicious documents against a set of source documents to generate a plagiarism score and a ranked list of matched sources with similarity percentages. It is treated as an external dependency by the web platform, integrated via HTTP calls triggered by Celery background tasks.

The two components communicate over an HTTP API. This separation allows each component to be developed, tested, and updated independently, and allows the AI service to be replaced or upgraded without modifying the web application.

1.5 Report Organization

The remainder of this report is structured as follows:

- Chapter 1 — Introduction
- Chapter 2 — Fundamental Concepts and Literature Review
- Chapter 3 — Project Management

- Chapter 4 — Requirements and System Analysis
- Chapter 5 — System Design
- Chapter 6 — Implementation
- Chapter 7 — Testing and Evaluation
- Chapter 8 — Conclusion and Future Work

1.6 Summary

This chapter has introduced the motivation and objectives of the Generative Plagiarism Detection project. The system addresses a real and growing need in academic institutions by providing a structured, asynchronous, and extensible web platform for managing AI-generated plagiarism detection. The AI analysis logic is cleanly encapsulated in a separate microservice, keeping the main application focused on workflow and user experience. The following chapters present the technical and managerial foundations of the project in detail.

Chapter 2: Fundamental Concepts and Literature Review

2.1 Introduction

This chapter provides a foundation for understanding the Generative Plagiarism Detection system by exploring the fundamental concepts and reviewing related studies. It examines essential terminology, and the principles behind the technologies used in the system. The aim is to present an overview of AI applications in source retrieval and document analysis, along with insights from existing detection solutions to guide and support the development of the proposed system.

2.2 Fundamental Concepts

This section explains the core concepts, terms, and theoretical principles relevant to the system, including:

- **Web Application Development (Django & React):** An overview of full-stack web development and how modern frameworks enable structured, maintainable, and scalable applications.
- **Asynchronous Task Processing (Celery & RabbitMQ):** An explanation of how long-running operations — such as document analysis — are handled outside the HTTP request-response cycle to preserve application responsiveness. Celery is a distributed task queue for Python that allows the Django application to delegate time-consuming jobs to background worker processes. RabbitMQ serves as the message broker, relaying task messages between the application and the workers.
- **Microservice Architecture and API Integration:** discussion of how decomposing an application into independently deployable services improves maintainability and flexibility. In this project, the AI detection logic is encapsulated in a separate FastAPI microservice, while the main web platform handles user-facing concerns. The two communicate via a REST API — a widely adopted architectural style for inter-service communication based on standard HTTP methods and JSON data exchange.

- **Object Storage (MinIO):** An introduction to object storage and its role in managing unstructured data such as uploaded documents and source files.
- **Generative Plagiarism:** A discussion of the specific form of academic dishonesty this system is designed to detect. Generative plagiarism occurs when a language model synthesizes a new document from existing academic papers, producing text that is semantically derived from the sources but shares no verbatim overlap with them. This makes conventional string-matching and fingerprinting detection tools fundamentally ineffective, representing a growing challenge for academic institutions as large language models become more accessible.
- **Source Retrieval:** An explanation of the detection strategy adopted in this project. Rather than comparing text directly, source retrieval treats detection as an information retrieval task: given a suspicious document, the system identifies and ranks the original academic papers most likely used to generate it. Key concepts include document embeddings, dense and sparse retrieval, hybrid fusion, and similarity scoring — all of which underpin the AI microservice component of the system.

2.3 Literature Review of Existing Systems

This section analyzes and evaluates existing plagiarism detection platforms in relation to the proposed system. The analysis focuses on how each platform handles document submission, user roles, analysis workflows, and result presentation — the core software concerns of this project. Four widely used platforms are reviewed:

Turnitin

Turnitin is the most widely adopted academic plagiarism detection platform globally. It compares submitted documents against a large reference database. In 2023, Turnitin introduced an AI writing detection module (Turnitin Originality) capable of identifying content generated by large language models such as ChatGPT.

Advantages:

1. Largest reference database in the industry, 1.9 billion student papers, archived web content, and premium journal subscriptions.
2. Deep LMS(Learning Management System) integration via with Canvas, Moodle, Blackboard, Microsoft Teams, and others.
3. Comprehensive toolset: similarity detection, AI writing detection, inline grading, and peer review.
4. Supports 176 languages for similarity detection; AI detection currently strongest in English.
5. Detailed similarity report with color-coded match groups and source links.

Disadvantages:

1. Expensive institutional subscription model, pricing is not publicly disclosed and varies significantly depending on the institution.
2. AI writing detection is a paid add-on, not included by default.
3. AI detection identifies AI-generated text in general but cannot identify which specific source papers were used to generate it.
4. False positive concerns: some genuine student work has been incorrectly flagged as AI-generated.
5. No open-source or self-hosted deployment option; all data is processed on Turnitin's servers.

iThenticate

iThenticate, also operated by Turnitin, is designed for researchers, publishers, and professional authors rather than students. It checks manuscripts, grant proposals, theses and dissertations against Turnitin's academic content database and web sources.

Advantages:

1. Targeted at academic publishing workflows, integrates with major Manuscript Tracking Systems (ScholarOne, Editorial Manager).
2. Private submitted-works repository allows publishers to detect collusion or duplicate submissions across their own archive.
3. AI writing detection available as add-on, including AI paraphrasing detection.
4. REST API available for integration into publishing pipelines.
5. Supports multiple languages and file formats including PDF and DOCX.

Disadvantages:

1. No student-facing submission workflow, not designed for classroom or course management use.
2. AI detection identifies AI-generated text generally but cannot attribute it to specific source papers.
3. High cost, minimum pricing makes it inaccessible for individual researchers or small institutions.
4. AI detection is a paid add-on, not included in the base product.

Copyleaks

Copyleaks is a cloud-based content integrity platform offering both traditional plagiarism detection and AI-generated content detection. It supports detection across 30+ languages and covers content generated by leading LLMs including ChatGPT, Gemini, DeepSeek, and Claude. Independent third-party studies have consistently ranked Copyleaks among the most accurate AI detection tools available.

Advantages:

1. Combined plagiarism and AI detection in a single platform, covering 30+ languages.
2. Detects AI-generated and AI-paraphrased content from all major LLMs.
3. REST API and LMS integration for seamless workflow embedding.
4. Strong security and compliance certifications.
5. Code plagiarism detection module for programming assignments.

Disadvantages:

1. Credit-based pricing can be unpredictable and costly at high volume.
2. AI detection and plagiarism detection were historically separate modules, integration is still maturing.
3. Accuracy of AI detection can fluctuate, some independent tests have shown significant variability.
4. No source-level retrieval, detects that a document is AI-generated but cannot identify which papers it was derived from.

Originality.AI

Originality.AI is a web-based platform combining AI-generated content detection with plagiarism checking, primarily targeting content publishers, agencies, educators, and SEO professionals. In May 2025, it released Plagiarism Checker Version 2, featuring improved paraphrase detection accuracy and broader web coverage.

Advantages:

1. Consistently ranked among the most accurate AI detection tools in independent studies.
2. Combined AI detection and plagiarism checking in one dashboard.
3. Pay-as-you-go pricing model is cost-effective for variable usage.
4. API access available for integration into content workflows and educational platforms.
5. Team management features with role assignment (Admin, Manager, User) suited for organizational use.

Disadvantages:

1. High false positive rate on human-written text.
2. Plagiarism detection checks primarily against web content via Google; not connected to a dedicated academic journal database comparable to Turnitin or iThenticate.
3. No dedicated student submission or course management workflow.
4. No detection of which source papers were used to generate AI text, only flags the presence of AI-generated content.

In conclusion, the four reviewed platforms provide solid foundations for traditional plagiarism detection and, increasingly, AI-generated content detection. However, none of them address generative plagiarism through semantic source retrieval, the ability to identify which specific academic papers were used to generate a submitted document. Additionally, none offer an open-architecture, self-hosted platform that cleanly separates detection logic from the user-facing application. The proposed system fills this gap by combining a structured web platform with a dedicated AI microservice focused on source retrieval, making both components independently maintainable and extensible.

Table 1: Comparative Analysis of Similar Systems and the Target System

Feature	Turnitin	iThenticate	Copyleaks	Originality.AI	Our System
Document Upload & Submission Management	✓	✓	✓	✓	✓
Similarity / Plagiarism Detection	✓	✓	✓	✓	✓
AI-Generated Content Detection	✓ ¹	✓ ¹	✓	✓	✓
Generative Plagiarism / Source Retrieval					✓
Ranked Source Attribution					✓
Asynchronous Background Processing	✓	✓	✓		✓
Detailed Match Report with Source Links	✓	✓	✓	✓	✓
Open Source					✓

(¹) Partial support: AI detection available but does not identify specific source papers.

2.4 Literature Review of AI Service

This section reviews four research approaches submitted to the PAN 2025 Generative Plagiarism Detection shared task, alongside the official task overview paper that established the evaluation baselines. The PAN 2025 task introduced a new benchmark for detecting generative plagiarism at the passage level: given a suspicious document generated by a large language model from an academic source paper, participating systems must identify and align the plagiarized spans with their original source passages. The reviewed works represent the state of the art on this task and directly inform the retrieval and detection strategy adopted in the AI microservice component of the proposed system.

Mo et al. (2025) — Multi-Feature Fusion with Graph-Based Merging [1]

The winning team at PAN 2025 combined semantic similarity and lexical overlap features using GloVe static word embeddings, spaCy for tokenization and text segmentation, WordNet for lemmatization, and NLTK for stop-word removal. Candidate plagiarized spans were first identified by scoring text segments across both feature dimensions, then merged and refined using a graph-based boundary optimization algorithm with soft Non-Maximum Suppression

(soft-NMS) to produce clean, non-overlapping detections. A multi-threshold verification step filtered low-confidence candidates before final output. The system achieved a PlagDet score of 0.584 with a recall of 0.727 and a granularity of exactly 1.0, meaning each detected span was matched to exactly one source without fragmentation. The key limitation is reduced precision (0.487) resulting from the deliberate trade-off of casting a wider semantic net, the preset thresholds may also require retuning when applied to corpora outside PAN 2025.

Su et al. (2025) — Hierarchical Multi-Model Vector Retrieval [2]

Su et al. proposed a hierarchical similarity matching pipeline that ensembles three complementary representations: Sentence-BERT and MPNet transformer embeddings for dense semantic similarity, and TF-IDF for sparse lexical weighting. Candidate spans are retrieved using FAISS for high-speed vector search, then re-ranked by cosine similarity across the ensemble. A BERT-based pairwise classifier serves as a fallback verification stage for borderline cases, and a block merging algorithm consolidates adjacent detections. This architecture achieved a PlagDet score of 0.496 with notably strong precision (0.600) relative to recall (0.578), reflecting the benefit of the BERT verification stage in filtering false positives. The granularity of 1.275 indicates some over-segmentation. The primary limitation is that recall is bounded by the top-k candidate pool retrieved by FAISS, creating an inherent speed-recall trade-off that the authors acknowledge.

Luo et al. (2025) — Two-Stage TF-IDF/Jaccard Filtering with BERT Classification [3]

Luo et al. adopted a two-stage pipeline designed to minimize computational cost. In the first stage, TF-IDF cosine similarity followed by character 3-gram Jaccard filtering aggressively narrows the candidate space, retaining only the most lexically similar sentence pairs for further analysis. In the second stage, a fine-tuned BERT classifier makes the final plagiarism decision on the surviving candidates. While the architecture demonstrates strong computational efficiency and achieves reasonable precision (0.582), it suffers from very low recall (0.177), indicating that the aggressive lexical filtering in stage one discards a large proportion of genuinely plagiarized spans that share no surface-level overlap with their sources, precisely

the defining characteristic of generative plagiarism. The authors also trained the BERT classifier on only 10,000 sentence pairs out of a possible 2.4 million, which likely constrained its ability to generalize across diverse obfuscation patterns.

Tang et al. (2025) — E5 Sentence Embeddings with FAISS Retrieval [4]

Tang et al. built a retrieval pipeline centered on the `intfloat/e5-base-v2` sentence transformer model, encoding both suspicious and source documents into dense vector representations and retrieving candidates using FAISS approximate nearest-neighbor search. A sliding window mechanism is applied to segment documents into overlapping chunks before encoding, improving coverage of plagiarized spans that cross sentence boundaries. The system achieved strong validation results (Recall: 0.73, Precision: 0.70, F1: 0.71), though precision degraded significantly on the held-out test set (0.33), suggesting overfitting to validation thresholds. The authors note that adding a cross-encoder reranking stage could substantially improve precision.

Greiner-Petter et al. (2025) — PAN 2025 Task Overview and Official Baselines [5]

The task overview paper by Greiner-Petter et al. establishes three semantic similarity baselines using large decoder-only language models (Linq-Embed-Mistral, Qwen, and Llama) alongside a lexical n-gram baseline derived from the PAN 2012 benchmark. Strikingly, the Linq-Embed-Mistral baseline outperformed most team submissions with a PlagDet score of 0.610, a recall of 0.820, and a precision of 0.580, underscoring how competitive strong embedding models are when applied directly to this task. The overview also documents a consistent performance gap on the older PAN 2012 dataset, attributed to its focus on sentence-level rather than paragraph-level plagiarism. The baselines establish a meaningful reference point against which the retrieval approach of the proposed AI microservice is evaluated, and confirm that dense embedding-based retrieval is the most competitive general strategy for this task.

Table 2: Comparative Analysis of Plagiarism Detection Approaches at PAN 2025

Paper	Approach	Embedding Model	Key Tools	Results	Key Strength	Key Limitation
Mo et al. [1]	Multi-feature fusion (semantic + lexical), graph-based merging, soft-NMS boundary refinement	GloVe (static word embeddings)	spaCy, WordNet, NLTK	PlagDet: 0.584 Recall: 0.727 Precision: 0.487	Winner at PAN 2025; optimal granularity (1.0)	Reduced precision (0.487); preset thresholds need tuning
Su et al. [2]	Hierarchical multi-model ensemble (SBERT + MPNet + TF-IDF), BERT pairwise fallback, block merging	SBERT, MPNet (transformer); TF-IDF (statistical)	spaCy, FAISS	PlagDet: 0.496 Recall: 0.578 Precision: 0.600	BERT fallback improves precision; strong multi-model ensemble	Recall bounded by FAISS top-k; speed-recall trade-off
Luo et al. [3]	Two-stage: TF-IDF cosine + Jaccard filtering, then fine-tuned BERT classifier	TF-IDF (statistical); BERT (fine-tuned classifier)	NLTK	Recall: 0.177 Precision: 0.582	High computational efficiency; reduced search space	Very low recall (0.177); lexical filtering discards semantic matches
Tang et al. [4]	Document-level filtering, sentence embedding + FAISS retrieval, sliding window chunking	intfloat/e5-base-v2 (sentence transformer)	FAISS	Recall: 0.73 Precision: 0.700	High recall via sliding window + e5 embeddings	Precision drops on test set (0.33); threshold overfitting
Greiner-Petter et al. [5]	Official baselines: semantic similarity (decoder LLMs) + lexical n-gram (PAN 2012)	Linq-Embed-Mistral, Qwen, Llama (decoder LLMs)	—	PlagDet: 0.610 Recall: 0.820 (Linq)	Linq baseline outperforms most team submissions; establishes strong upper bound	Performance drop on PAN 2012 (sentence-level dataset)

Across the reviewed approaches, two consistent findings emerge:

- Dense semantic retrieval using transformer-based embeddings substantially outperforms purely lexical methods in recall, a critical observation given that generative plagiarism by definition produces text with no verbatim overlap with its

sources. The failure of Luo et al.'s [3] lexical-first filtering to achieve meaningful recall (0.177) directly confirms this.

- Precision is the harder problem: even the best teams struggle to avoid false positives, and the most effective countermeasure identified in the literature is a secondary verification stage, either a BERT-based classifier (Su et al [3].) or boundary refinement (Mo et al [1].). The proposed AI microservice draws directly from these findings: it adopts e5-base-v2 dense embeddings (following Tang et al [4].), augments them with BM25 sparse retrieval in a hybrid fusion scheme to capture both semantic and lexical signals, and uses the same sliding window technique for chunking from [4]. A cross-encoder reranking stage is planned as a future improvement to address the precision gap identified across all reviewed systems.

2.5 Summary

This chapter introduced the core technologies underpinning the system, defined generative plagiarism as the specific form of misconduct the system targets, and reviewed both existing detection platforms and state-of-the-art AI approaches from PAN 2025. The findings confirmed that dense embedding-based retrieval is the most effective strategy for this task, directly informing the design of the AI microservice. The following chapter presents the project management framework.

Chapter 3: Project Management

3.1 Introduction

This chapter presents the project management framework governing the Generative Plagiarism Detection project. It covers the project charter, team roles and responsibilities, the Statement of Work (SOW), project scope and objectives, deliverables, requirements, assumptions, resources, development schedule, and risk management plan. A clear and well-documented project management structure is essential for a two-person graduation project where each member owns distinct technical areas that must integrate cleanly.

3.2 Project Charter

The project charter formally authorizes the project and defines its high-level parameters, team composition, and success criteria.

Field	Details
Project Title	Generative Plagiarism Detection System
Project Type	Junior Project
Academic Year	2025 – 2026
Project Start Date	Feb 15, 2026
Project Finish Date	May 10, 2026
Academic Supervisor	Dr. Riad Sonbol
Technical Supervisor	Eng. Raghad Alhossny
Project Purpose	Develop a web-based platform for detecting AI-generated plagiarism in academic submissions, integrating a full-stack web application with an AI analysis microservice.
Primary Deliverable	A fully functional, integrated web platform with documented source code, API contracts, and this thesis report.
Success Criteria	System correctly handles the end-to-end submission and analysis workflow; results are presented clearly to users, system passes integration testing, thesis report completed and approved.
Constraints	Two-person team, fixed academic semester timeline, limited computational resources for AI-side experiments.

3.3 Roles and Responsibilities

Role	Name	Responsibilities
Academic Supervisor	Dr. Riad Sonbol	Provides academic guidance and oversight, reviews project deliverables at key milestones, approves major scope decisions, and ensures alignment with project requirements.
Technical Supervisor	Eng. Raghad Alhossny	Provides technical guidance on system design, architecture decisions, and code quality throughout the project lifecycle.
Full-Stack Developer (Web Platform)	Tala Alkhateeb	Designs and implements the Django backend, React frontend, SQL server schema, Celery task queue, REST API design, and overall web platform architecture and integration.
AI Service Developer	Farah Alkayyem	Designs and implements the FastAPI AI microservice, document encoding pipeline, retrieval logic, /analyze API endpoint, and submits to TIRA for official evaluation.

3.4 Statement of Work (SOW)

The Statement of Work is a formal agreement that details the specific boundaries and technical requirements of a project. It serves as a contractual roadmap, defining exactly what is in-scope to prevent misunderstandings. The SOW ensures all parties stay aligned on the project's execution and final delivery.

3.4.1 Project Description and Objectives

The project delivers a Generative Plagiarism Detection system consisting of two integrated components: a full-stack web application and an AI analysis microservice. The web application enables students and instructors to upload, manage, and analyze academic documents for AI-generated plagiarism in a structured environment. The AI microservice handles the underlying detection logic and exposes its results through a documented REST API. The primary objective is to produce a working, integrated system demonstrating sound software engineering principles: separation of concerns, asynchronous processing, and clean API design.

3.4.2 Project Scope

- Full-stack web application: Django backend, React frontend, SQL server database, Celery task queue with Redis broker.

- AI analysis microservice: FastAPI service exposing /analyze endpoint, integrated with the web platform via HTTP.
- User management: registration, login, workspaces management.
- Document and source management: file upload, storage, and association with submissions.
- Asynchronous analysis workflow: task queuing, status tracking, result storage, and structured result display.
- Project documentation: thesis report, README files, and API contract specification.

3.4.3 Project Goals

- Deliver a functional, tested, and documented web platform for generative plagiarism detection.
- Achieve clean, verified integration between the web platform and the AI microservice.
- Demonstrate sound software engineering practices throughout the development process.
- Complete and submit the graduation thesis report covering all aspects of the system.

3.4.4 Deliverables

Deliverable	Status	Description
Project Charter & SOW	Completed	Formal initiation documents defining scope, objectives, roles, and responsibilities.
System Architecture Document	Completed	High-level design covering the web platform, AI microservice, and their integration contract.
Django Backend – Core Modules	Completed	User management, document/source models, submission workflow, Celery task setup, REST API endpoints.
React Frontend – Core Views	Completed	Login, dashboard, upload, submission history, and results display pages.
AI Microservice	Completed	FastAPI service returning plagiarism score and matched sources; integrated with vector store.
Web–AI Service Integration	Completed	Celery task calling /analyze endpoint.
End-to-End Testing	In Progress	Integration tests covering the full submission-to-result workflow across both components.
Final Thesis Report	In Progress	Complete graduation project report covering all chapters.
Final Presentation & Demo	Date TBD	Oral defense with live system demonstration.

3.4.5 Assumptions

- Both team members will remain available and active throughout the project duration.
- The AI microservice /analyze endpoint will be finalized before the web integration deadline; a mock fallback is implemented in the Django backend in case of delays.
- Supervisors will be available for regular check-ins and will provide timely feedback on deliverables at each milestone.
- Development is performed on personal laptops, GPU-enabled cloud resources (Google Colab) are available for AI-side experiments.
- All adopted frameworks and tools (Django, React, SQL server, Celery, Redis, FastAPI) are used under their open-source licenses with no associated licensing cost.
- The system will be evaluated in a local or staging environment, production-scale deployment is not required for the project assessment.

3.4.6 Project Resources

Human Resources:

- Tala Alkhateeb — full-stack web development
- Farah Alkayyem — AI service development
- Dr. Riad Sonbol — academic supervision
- Eng. Raghad Alhossny — technical supervision

Technical Resources:

- Personal development laptops for web application development and local testing.
- Google Colab with GPU access for AI model encoding experiments.
- GitHub for version control, collaboration, and code review.
- Docker for containerizing and portably running the services.
- TIRA platform for official AI retrieval evaluation and benchmarking submissions.

3.4.7 Schedule

Phase	Timeline	Key Activities
Initiation and Study	Feb 9 – Feb 18	Literature review, SRS documentation
Analysis and Design	Feb 19 – March 8	Diagrams, Architecture design

Phase	Timeline	Key Activities
Implementation	March 9 – March 17	Frontend + Backend setup, AI baseline
Integration	March 16 – March 25	Integration, Storage service, AI optimization
Advances Features	March 26 – April 3	Celery setup, AI advanced pipeline
Second Seminar	April 4 – April 27	System testing
Technical Interview	April 14 – May 11	UI Polishing, AI improvement
Final Delivery	April 20 – May 16	Thesis writing, Final testing

3.4.8 Risk Management

Risk	Probability	Impact	Mitigation Strategy
AI service /analyze endpoint not finalized on time, blocking web integration	Medium	High	Mock response mode implemented in Django backend so web platform development continues independently.
Celery worker failures causing lost analysis tasks	Low	High	Task result backend configured; retry policies defined on Celery tasks; task status exposed in the UI.
Team member unavailability due to personal circumstances	Low	Medium	All work version-controlled on GitHub, documentation maintained so either member can continue the other's work.
Integration failures between Celery tasks and the AI service	Medium	High	Integration tested early with a minimal end-to-end prototype before full feature development.
Resource Exhaustion (RAM/GPU) when running large embedding models	High	Medium	Use quantized or "base" model versions (e.g., e5-base-v2), optimize FAISS index memory usage.
Deployment challenges due to lack of experience	Medium	Medium	Research best practices

3.5 Summary

This chapter has presented the complete project management framework for the Generative Plagiarism Detection system. The project charter defines the project formally, identifying the two-student team, two supervisors, and key success criteria. Roles are clearly assigned between team members, with one focused on the web platform and the other on the AI service. The

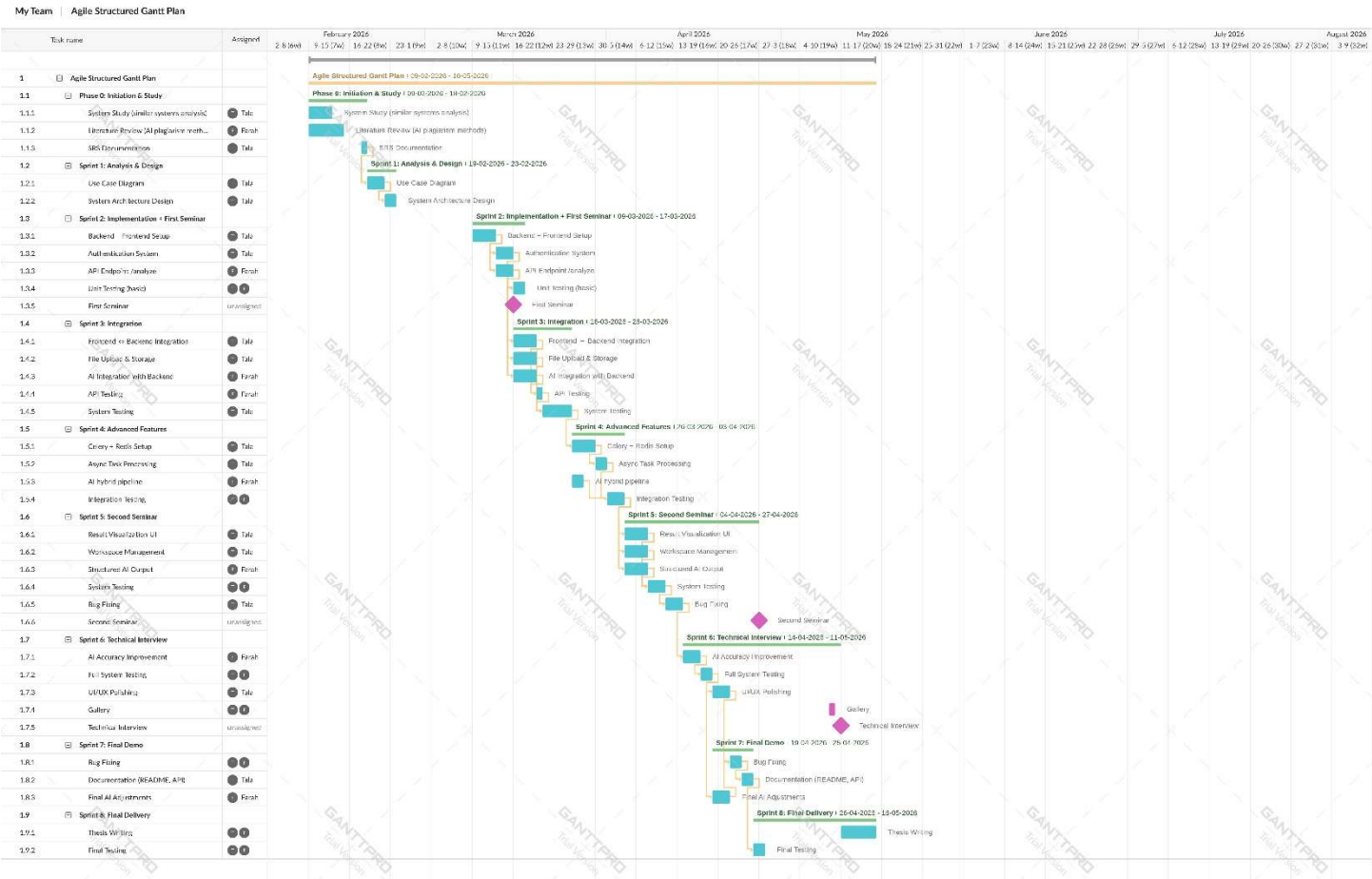
Statement of Work specifies scope boundaries, goals, deliverables, resource categories, an eight-phase schedule, and a risk management table covering identified risks with mitigation strategies. With this management foundation established, the following chapters address system requirements, design, and implementation in detail.

Chapter 4: Requirements and System Analysis

4.1 Introduction

This chapter focuses on the detailed analysis of the Generative Plagiarism Detection System. It defines the system's requirements, functionality, and architecture by examining user needs and the operational context in which the platform will be used. The chapter includes an exploration of the project timeline, functional and non-functional requirements, use cases, and system workflows to provide a comprehensive understanding of how the system addresses the identified challenges and achieves the project objectives. This analysis forms the foundation for designing and implementing the proposed solution.

4.2 Project Timeline



Software Requirement Specification Document (SRS)

1. Introduction

1.1 Purpose

This document specifies the software requirements for the Generative Plagiarism Detection system, a solution designed to detect and report generative plagiarism in academic submissions. The system addresses the inability of traditional plagiarism tools to identify AI-generated text that is semantically derived from existing academic source papers.

1.2 Project Scope

The GPDetect system automates and enhances academic integrity checking through semantic retrieval, focusing on detecting generative plagiarism. The specification covers high-level requirements and outlines the actors within the system.

1.2.1 High-Level Requirements:

- **User Management:** Allows administrators to manage user accounts by adding, updating, and deleting profiles. Implements role-based access control, automatically assigning roles (Regular User or Administrator) based on the registration process.
- **Subscription Plan Management:** The system provides configurable subscription plans with monthly check quotas, maximum source and document limits, and allowed file formats. Only administrators can create or modify plans.
- **Workspace Management:** Users can create, rename, and delete named workspaces. Each workspace groups source reference documents and suspicious documents together as a logical analysis unit.
- **File Upload and Storage:** The system supports upload of PDF, DOCX, DOC, and TXT files. Files are stored in MinIO object storage via the Storage Microservice; only the file key (storage path) is persisted in the database.

Asynchronous Analysis Submission: Users submit a workspace for analysis by selecting source documents (minimum 2) and suspicious documents. The system queues the job via Celery, calls the AI microservice, and processes results in the background without blocking the user interface.

- **AI-Powered Retrieval:** The AI microservice encodes documents into dense vectors using a sentence transformer model, stores them in a Qdrant vector database, and retrieves the most semantically similar source documents using a hybrid BM25 + dense retrieval scheme.
- **Results Display and Reporting:** The system presents per-document plagiarism scores (0–100%), ranked matched sources with colour-coded similarity percentages, and highlighted plagiarised text segments. Users can download a PDF report of their results.
- **Submission History:** View a paginated history of all past submissions across all workspaces, including submission date, status, and expandable per-document result cards.
- **Account Management:** Users can edit their profile, change their password, and delete their account. Administrators can view all accounts and activate or deactivate any user.

1.2.2 Actors:

Actor	Type	Description
Regular User	Human	Authenticated user on a subscription plan. Submits academic documents for analysis and reviews results.
Administrator	Human	Privileged operator who manages subscription plans, user accounts, and system configuration.

AI Microservice	System	FastAPI service that receives analysis requests, encodes documents, searches Odrant, and returns plagiarism scores.
Storage Microservice	System	FastAPI + MinIO service that stores uploaded files, generates presigned URLs, and persists analysis result JSON.

2. Overall Description

2.1 Purpose

The GPDetect system is a new, self-contained product designed to detect generative plagiarism in academic documents through semantic source retrieval. It is not part of an existing product family, nor is it a replacement for any current system in use at the institution.

2.2 Product Features

- **User Management:** Create, update, and delete user accounts. Role-based access control (Regular User, Administrator).
 - Administrators can view all registered accounts, searchable by name, email, role, and status.
 - Administrators can activate or deactivate any user account.
 - All users can edit their own name and email, change their password, and delete their account with password confirmation.
- **Authentication and Registration:** Two-step OTP registration flow, user verifies the code to complete account creation.
- **Subscription Plan Management:** Administrators can create, edit, and deactivate subscription plans.
- **Workspace Management:** Users can create, rename, and delete named workspaces. Each workspace is fully isolated per user.
- **File Upload — Sources and Documents:** Users upload source reference files and suspicious documents in PDF, DOCX, DOC, or TXT format.

- **Analysis Submission:** Users submit a workspace for analysis by selecting source document IDs (minimum 2) and suspicious document IDs (minimum 1).
- **AI-Powered Retrieval:** The AI Microservice processes submitted documents using a dense retrieval pipeline.
- **Results Display:** Each submitted document receives a plagiarism score (0–100%), an original percentage, and a ranked list of matched sources sorted by similarity descending.
 - Matched sources are color-coded by risk level.
 - Highlighted plagiarised text segments are displayed with source attribution and match percentage.
- **PDF Report Download:** Users can download a client-side PDF report directly in the browser with no additional server request.
- **Submission History:** A paginated history view shows all past submissions across all workspaces, ordered by date descending.

2.3 User Classes and Characteristics

- **Administrator:** Administrators are privileged platform operators. Their role is operational management of the user base and subscription structure, they do not use the system to check academic documents themselves.
 - **Responsibilities:**
 1. Log in with the same authentication flow as regular users (differentiated by role='admin' in the token payload).
 2. View the admin dashboard showing aggregate statistics: total users, active users and active plan count.
 3. Create, edit, and deactivate subscription plans.
 4. View the full list of registered user accounts.
 5. Activate or deactivate any user account, inactive users are blocked from logging in.

- **Privileges:**
 1. Full access to user management and plan management endpoints.
 2. Access to the admin dashboard showing aggregate platform statistics.
- **User:** Users are students, academics, or researchers who wish to verify the originality of their academic work. They are not required to have any technical knowledge of AI or information retrieval systems.
 - **Responsibilities:**
 1. Register an account by providing name, email, password, and a subscription plan, verify identity via OTP email code.
 2. Create, rename, and delete their own workspaces.
 3. Upload source reference files, and suspicious documents, within plan limits.
 4. Submit a workspace for analysis.
 5. View per-document plagiarism scores and download PDF report.
 6. Edit profile information, change password, and delete account.
 - **Privileges:**
 1. Full access to all workspace, file upload, submission, and results features within the scope of their own account and plan limits.
 2. Access to account self-management: profile editing, password change, and account deletion.

3. System Features

3.1 Functional Requirements

3.1.1 User Management:

- **REQ-1.1:** The system shall allow new users to register by providing their name, email address, password, and a selected subscription plan.
- **REQ-1.2:** The system shall send a 6-digit One-Time-Password (OTP) to the supplied email address upon completion of the registration form.
- **REQ-1.3:** The system shall verify the OTP code entered by the user. OTP codes shall expire after 10 minutes.
- **REQ-1.4:** The system shall create the user account and return JWT access and refresh tokens upon successful OTP verification.
- **REQ-1.5:** The system shall allow the Administrator to view all registered user accounts, searchable and filterable by name, email, role, and status.
- **REQ-1.6:** The system shall allow the Administrator to change any user's account status between 'active' and 'inactive'.
- **REQ-1.7:** The system shall allow all users to edit their own account information (name, email and password) via the profile page.
- **REQ-1.8:** The system shall allow users to delete their own accounts by confirming with their current password. Deletion shall cascade to all owned workspaces and results.

3.1.2 Plan Management:

- **REQ-2.1:** The system shall allow anyone (authenticated or not) to retrieve the list of available subscription plans to support the registration plan-selection page.
- **REQ-2.2:** The system shall allow the Administrator to create a new subscription plan specifying: name, monthly price,

checks_per_month quota, max_sources, max_documents, and allowed_formats.

- **REQ-2.3:** The system shall allow the Administrator to update an existing plan's parameters, or deactivate it.

3.1.2 Workspace Management:

- **REQ-3.1:** The system shall allow authenticated users to create a new workspace by providing a name. Each workspace is scoped exclusively to the creating user.
- **REQ-3.2:** The system shall allow users to view a list of their own workspaces, including source count, document count, and current status.
- **REQ-3.3:** The system shall allow users to rename an existing workspace.
- **REQ-3.4:** The system shall allow users to delete a workspace. Cascade deletion shall remove all associated sources, documents, submissions, and results.
- **REQ-3.5:** The system shall automatically manage workspace status:
 - **REQ-3.5.1:** Status is set to 'draft' on workspace creation.
 - **REQ-3.5.2:** Status is updated to 'pending' when an analysis submission is queued.
 - **REQ-3.5.3:** Status is updated to 'analyzed' when analysis completes successfully.

3.1.4 File Upload:

- **REQ-4.1:** The system shall allow users to upload source reference files to a workspace. Accepted formats are: PDF, DOCX, DOC, TXT. Files shall be stored in MinIO via the Storage Microservice.

- **REQ-4.2:** The system shall enforce the plan's max_sources limit per workspace.
- **REQ-4.3:** The system shall allow users to delete a source file.
- **REQ-4.4:** The system shall allow users to upload suspicious documents to a workspace using the same format restrictions and storage pattern as sources.
- **REQ-4.5:** The system shall enforce the plan's max_documents limit per workspace.
- **REQ-4.6:** The system shall allow users to delete a suspicious document from a workspace.

3.1.5 Analysis Submission and Processing:

- **REQ-5.1:** The system shall allow users to submit a workspace for analysis by selecting source document IDs (minimum 2) and suspicious document IDs (minimum 1).
- **REQ-5.2:** The system shall verify the user's plan quota before queuing.
- **REQ-5.3:** The system shall verify document format validity before queuing.
- **REQ-5.4:** On a valid submission, the system shall create a Submission record with status 'pending' and dispatch an asynchronous Celery task to the message broker.
- **REQ-5.5:** The Celery task shall call POST /analyze on the AI Microservice, providing the document presigned URL and a list of source presigned URLs.
- **REQ-5.6:** The system shall poll the Storage Microservice for the analysis result at 3-second intervals for a maximum of 120 seconds.
- **REQ-5.7:** Upon receiving a valid result, the system shall persist a DocumentResult record (plagiarism score, original percentage, highlighted segments) and MatchedSource records (one per source, sorted by match percentage descending).

- **REQ-5.8:** The system shall update the submission status to 'completed' and the workspace status to 'analyzed' upon successful processing.
- **REQ-5.9:** Failed Celery tasks shall set the submission status to 'failed'. The task shall retry up to 3 times with a 60-second countdown between attempts.
- **REQ-5.10:** The AI Microservice shall process each analysis request as a background task. The microservice shall:
 - **REQ-5.10.1:** Download source bytes from presigned URLs; extract plain text via pdfplumber (PDF), python-docx (DOCX), or UTF-8 decode (TXT).
 - **REQ-5.10.2:** Chunk each source document using a sliding window (WINDOW_SIZE=6 sentences, STRIDE=2); encode chunks with e5-base-v2 (passage: prefix, batch_size=64, normalized embeddings); upsert to the Qdrant collection. Already-indexed sources shall be skipped.
 - **REQ-5.10.3:** Chunk and encode the suspicious document (query: prefix); search Qdrant per query chunk; aggregate chunk scores to document-level scores; rank documents descending; POST the result JSON to the callback URL.

3.1.6 Results Display and Reporting:

- **REQ-6.1:** The system shall provide an endpoint that returns the latest submission results for a workspace, including submission status and an array of per-document result objects.
- **REQ-6.2:** Each document result shall include: document name, plagiarism score (0–100%), original percentage, matched sources sorted by match percentage descending, and highlighted text segments.

- **REQ-6.3:** The system shall allow users to download a PDF report of their results.

3.1.7 Submission History:

- **REQ-7.1:** The system shall provide a submission history endpoint returning all submissions belonging to the authenticated user, ordered by date descending.
- **REQ-7.2:** Each history entry shall include: workspace name, submission date, status, and an embedded array of per-document result objects.

3.1.7 Authentication:

- **REQ-8.1:** The system shall authenticate users via email and password. On success it shall return a JWT access token (60-minute lifetime) and a refresh token (7-day lifetime).
- **REQ-8.2:** The system shall allow token refresh using a valid refresh token, returning a new access token.
- **REQ-8.3:** The system shall allow OTP resend, invalidating all previous unused OTPs for the email before generating a new code.

3.2 Non-Functional Requirements

3.2.1 Security:

- **NF-1.1:** The system shall enforce secure authentication, requiring users to log in using strong passwords.
- **NF-1.2:** The system shall encrypt all sensitive data (e.g., passwords, user information).

3.2.2 Scalability:

- **NF-2.1:** The Qdrant vector store shall be deployable as a standalone service.

- **NF-2.2:** The system shall be scalable to support an increasing number of users and analysis jobs without performance degradation.

3.2.3 Availability:

- **NF-3.1:** Celery task failures shall not result in silent data loss. Failed submissions shall be marked 'failed' and visible to the user in the workspace and history views.
- **NF-3.2:** The React SPA shall be fully responsive for screen widths from 375 px (mobile) through desktop.

3.2.4 Extensibility:

- **NF-4.1:** The AI microservice shall be designed so that alternative retrieval models can be integrated without modifying the web platform or the /analyze API contract.

4. System Requirements

Req-ID	Requirement	Category	Priority
REQ-1.1	The system shall allow new users to register by providing name, email, password, and plan.	User Management	High
REQ-1.2	The system shall send a 6-digit OTP to the user's email upon registration form completion.	User Management	High
REQ-1.3	The system shall verify the OTP code. OTP codes expire after 10 minutes.	User Management	High
REQ-1.4	The system shall create the user account and return JWT upon successful OTP verification.	User Management	High
REQ-1.5	The system shall allow the Administrator to view, search, and filter all registered user accounts.	User Management	High
REQ-1.6	The system shall allow the Administrator to change any user's account status (active/inactive).	User Management	High
REQ-1.7	The system shall allow all users to edit their own account information and change their password.	User Management	Medium

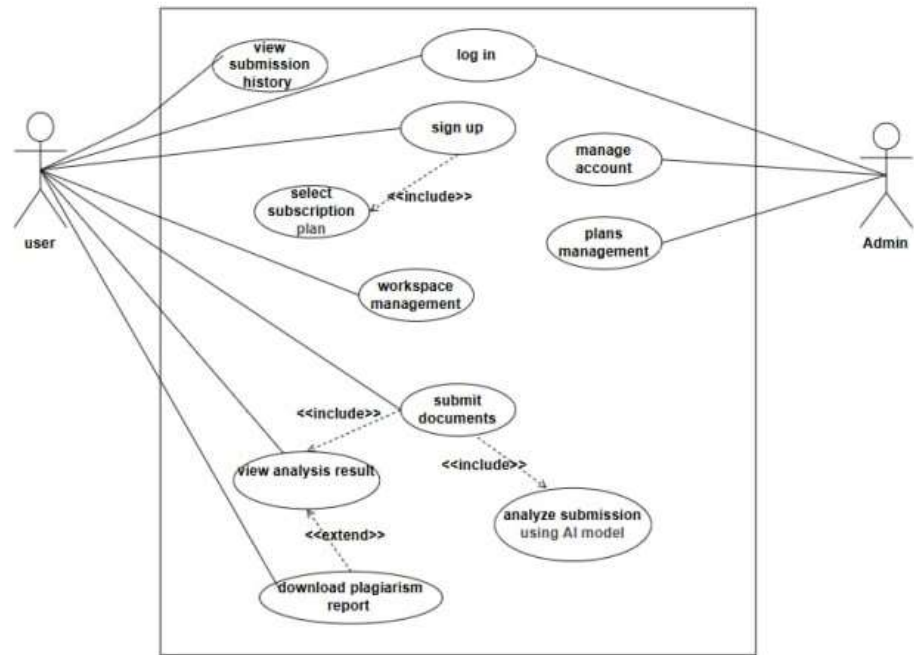
REQ-1.8	Users shall be able to delete their own accounts with password confirmation.	User Management	Medium
REQ-2.1	GET /api/plans/ shall be publicly accessible for the registration plan-selection page.	Plan Management	High
REQ-2.2	The system shall allow the Administrator to create a new subscription plan with name, price, quotas, and format limits.	Plan Management	High
REQ-2.3	The system shall allow the Administrator to update and deactivate existing plans.	Plan Management	High
REQ-3.1	The system shall allow users to create named workspaces scoped exclusively to the creating user.	Workspace Mgmt	High
REQ-3.2	The system shall allow users to list their own workspaces.	Workspace Mgmt	High
REQ-3.3	The system shall allow users to rename an existing workspace	Workspace Mgmt	Low
REQ-3.4	The system shall allow users to delete a workspace and all its associated details.	Workspace Mgmt	High
REQ-3.3	The system shall automatically manage workspace status: draft → pending → analyzed.	Workspace Mgmt	High
REQ-4.1	The system shall allow users to upload source reference files (PDF, DOCX, DOC, TXT) to a workspace.	File Upload	High
REQ-4.2	The system shall enforce the plan's max_sources limit.	File Upload	High
REQ-4.3	The system shall allow users to delete a source file.	File Upload	High
REQ-4.4	The system shall allow users to upload suspicious documents.	File Upload	High
REQ-4.5	The system shall enforce the plan's max_documents limit.	File Upload	High
REQ-4.6	The System shall allow users to delete a suspicious document.	File Upload	High
REQ-5.1	Users shall submit a workspace for analysis (min 2 sources, min 1 document).	Submission	High
REQ-5.2	The system shall verify quota validity before queuing	Submission	High
REQ-5.3	The system shall verify document format validity before queuing	Submission	High

REQ-5.4	A Celery task shall call /analyze, poll for results, and persist DocumentResult and MatchedSource records.	Submission	High
REQ-5.4	Failed tasks shall set submission status to 'failed' and retry up to 3 times with 60-second intervals.	Submission	High
REQ-5.5	The AI Microservice shall chunk, encode, and upsert source documents to Qdrant; retrieve against the suspicious doc.	Submission	High
REQ-6.1	The system shall provide an endpoint that returns the latest submission results for a workspace.	Results Display	High
REQ-6.2	The system shall return per-document results: score, original %, ranked sources with color codes, highlighted segments.	Results Display	High
REQ-6.3	Users shall be able to download a client-side PDF report of their analysis results.	Results Display	High
REQ-7.1	GET /api/submissions/history/ shall return all submissions of the authenticated user, ordered by date descending.	History	High
REQ-7.2	Each history entry shall include workspace name, date, status, and embedded per-document result objects.	History	Medium
REQ-8.1	Login shall return JWT access (60 min) and refresh (7 day) tokens.	Authentication	High
REQ-8.2	The system shall support token refresh and refresh-token blacklisting on logout.	Authentication	High
REQ-8.3	All protected endpoints shall require a valid JWT Bearer token.	Authentication	High
NF-1.1	The system shall enforce secure authentication, requiring users to log in using strong passwords.	Security	High
NF-1.2	The system shall encrypt all sensitive data (e.g., passwords, user information).	Security	High
NF-2.1	The Qdrant vector store shall be deployable as a standalone service.	Scalability	Medium
NF-2.2	The system shall be scalable to support an increasing number of users and analysis jobs without performance degradation.	Scalability	High
NF-3.1	Celery task failures shall not result in silent data loss. Failed submissions shall be marked 'failed' and visible to the user in the workspace and history views.	Availability	Medium
NF-3.2	The React SPA shall be fully responsive for screen widths from 375 px (mobile) through desktop.	Availability	High

NF-4.1	The AI microservice shall be designed so that alternative retrieval models can be integrated without modifying the web platform or the /analyze API contract.	Extensibility	High
---------------	---	---------------	------

5. Requirements Modeling

5.1 UML Diagram



Use Case ID	UC-01
Use Case Name	Sign Up
Actor	User
Description	Allows a new user to create an account by providing their personal information, selecting a subscription plan, and verifying their email address via a one-time password (OTP).
Preconditions	The user does not have an existing account with the same email.

Postconditions	• A new user account is created with status active and role user. • The user is automatically logged in and redirected to the Dashboard. • A JWT access token and refresh token are issued and stored.
Main Flow	1. The user navigates to the registration page. 2. The user enters full name, email address, password, and confirm password. 3. The user clicks continue. 4. The system validates the entered information. 5. The user selects a subscription plan (<i>include: UC-07</i>). 6. The user clicks send verification code. 7. The system Generates a 6-digit OTP, stores it, and sends it to the provided email. 8. The user enters the received OTP in the verification field. 9. The user clicks verify & create account. 10. The system creates the user account and marks the email as verified. 11. The system issues JWT tokens and redirects the user to the dashboard.
Alternative Flows	A1 – Email already registered: • 4.1 If the entered email already exists in the system, the system displays: <i>"An account with this email already exists."</i> • 4.2 The use case returns to step 2. A2 – Passwords do not match: • 4.1 If the password and confirm password fields do not match, the system displays: <i>"Passwords do not match."</i> • 4.2 The use case returns to step 2. A3 — Password too short: • 4.1 If the password is fewer than 6 characters, the system displays: <i>"Password must be at least 6 characters."</i> • 4.2 The use case returns to step 2. A4 – Invalid or expired OTP:

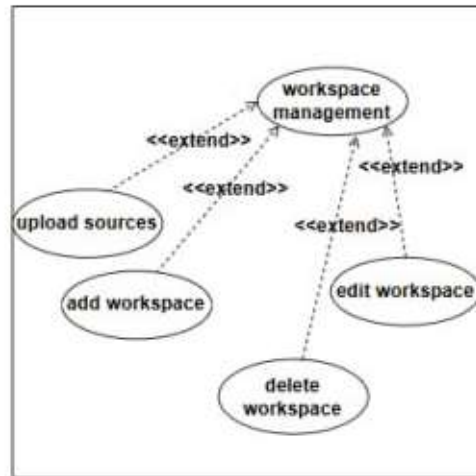
	10.1 If the entered OTP does not match or has expired (after 10 minutes), the system displays: <i>"Invalid or expired verification code."</i> 10.2 The use case returns to step 8. A5 – User requests OTP resend: 8.1 If the user clicks Resend Code after the countdown expires, the system invalidates the previous OTP, generates a new one, and sends it to the same email. 8.2 The use case returns to step 8.
--	---

Use Case ID	UC-01a
Use Case Name	Select Subscription Plan
Actor	User
Description	Allows a new user during the registration process to browse available subscription plans and select one that suits their needs. This use case is always executed as part of the sign-up flow before OTP verification is triggered.
Preconditions	- The user has completed and validated the registration form (steps 1–4 of UC-01). - At least one active subscription plan exists in the system.
Postconditions	- A subscription plan is selected and associated with the pending registration data. - The selected plan ID is passed back to UC-01 to be used during account creation at step 10.
Included by	UC-01 — Sign Up
Main Flow	- The system retrieves all active subscription plans from the database. - The system displays the available plans with name, price, and monthly check limit. - The user reviews the available plans and clicks on a plan to select it. - The system highlights the selected plan and stores the plan ID in the registration state.

	5. The system returns control to UC-01 to proceed to step 6 (Send verification code).
Alternative flows	A1 - No plan selected: 5.1 If the user clicks Send Verification Code without selecting a plan, the system displays: <i>"Please select a subscription plan."</i> 5.2 The use case returns to step 3.

Use Case ID	UC-02
Use Case Name	Log In
Actor	User, Admin
Description	Allows a registered user or admin to authenticate into the system using their email and password in order to access platform features.
Preconditions	- The user/admin has a registered account in the system - The account is in active status - The user/admin is not already logged in
Postconditions	- The user/admin is authenticated and redirected to Dashboard - A JWT access token and refresh token are issued and stored
Main Flow	- The user/Admin navigates to the Login page - The user/Admin enters email address and password - The user/Admin clicks the login button - The system validates the credentials against the database - The system issues a JWT access token and refresh token - The system redirects the user/admin to the dashboard
Alternative Flows	A1 – Invalid credentials: 4.1 If the email does not exist or the password is incorrect, the system displays: <i>"Invalid email or password"</i> 4.2 The use case returns to step 2

	A2 – Empty fields: 3.1 If either field is empty, the system displays: <i>“Please fill in all fields.”</i> 3.2 The use case returns to step 2.
--	---



Use Case ID	UC-03
Use Case Name	Workspace Management
Actor	User
Description	Allows an authenticated user to manage their workspaces. The user can create, edit, delete workspaces, and upload source files. This is the base use case extended by more specific workspace operations.
Preconditions	The user is authenticated and has an active account.
Postconditions	The requested workspace operation is completed and reflected in the system.
Extended by	UC-03a, UC-03b, UC-03c, UC-03d
Main Flow	1. The user Navigates to the workspaces page. 2. The system Retrieves and displays all workspaces belonging to the user.

	3. The user selects a workspace operation (add, edit, delete, or upload sources) 4. The system Executes the selected operation via the appropriate extended use case.
--	--

Use Case ID	UC-03a
Use Case Name	Add Workspace
Actor	User
Description	Allows an authenticated user to create a new workspace by providing a name. The workspace serves as a container for source files and documents to be analyzed.
Preconditions	The user is authenticated and has an active account.
Postconditions	A new workspace is created and appears in the user's workspace list.
Extends	UC-03 — Workspace Management
Main Flow	1. The user clicks the new workspace button. 2. The system displays the create workspace modal. 3. The user Enters a workspace name. 4. The user clicks create. 5. The system creates the workspace and saves it to the database. 6. The system closes the modal and redirects the user to the new workspace's detail page.

Use Case ID	UC-03b
Use Case Name	Edit Workspace
Actor	User
Description	Allows and authenticated user to rename an existing workspace.

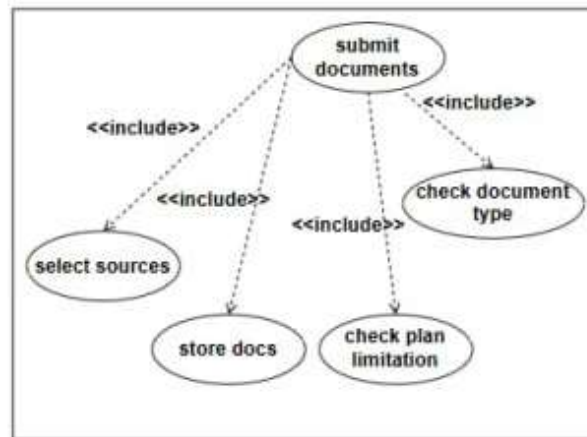
Preconditions	• The user is authenticated and has an active account. • At least one workspace exists.
Postconditions	The workspace is renamed and the updated name is reflected in the workspace list.
Extends	UC-03 — Workspace Management
Main Flow	1. The user clicks the edit icon on a workspace card. 2. The system displays the rename workspace modal pre-filled with the current name. 3. The user modifies the workspace name and clicks save. 4. The system updates the workspace name in the database. 5. The system closes the modal and reflects the new name in the workspace list.

Use Case ID	UC-03c
Use Case Name	Delete Workspace
Actor	User
Description	Allows an authenticated user to permanently delete a workspace along with all its associated sources, documents, and submissions.
Preconditions	• The user is authenticated and has an active account. • At least one workspace exists.
Postconditions	• The workspace and all its associated data are permanently deleted from the system. • The workspace no longer appears in the user's workspace list.
Extends	UC-03 — Workspace Management
Main Flow	1. The user clicks the delete icon on a workspace card. 2. The system displays a confirmation modal warning that all data will be permanently deleted.

	- The user clicks delete to confirm. - The system deletes the workspace and all associated data from the database and storage. - The system closes the modal and removes the workspace from the list.
--	---

Use Case ID	UC-03d
Use Case Name	Upload Sources
Actor	User
Description	Allows an authenticated user to upload source reference files to a workspace. These files will later be used by the AI model to compare against submitted documents during plagiarism analysis.
Preconditions	- The user is authenticated and has an active account. - A workspace exists and is open. - The user's plan allows additional sources (max sources limit not reached)
Postconditions	- The uploaded source file is stored in MinIO object storage. - The source appears in the workspace's sources list.
Extends	UC-03 — Workspace Management
Main Flow	- The user opens a workspace detail page. - The user clicks add sources or drags and drops a file into the sources drop zone. - The system validates the file format and plan limit. - The system Uploads the file to MinIO object storage via the storage microservice. - The system displays the uploaded file in the sources list with its name and size.
Alternative Flows	A1 – Unsupported file format: 3.1 If the file extension is not .pdf, .docx, .doc, or .txt, the system displays: <i>"Unsupported file type. Allowed: PDF, DOCX, DOC, TXT."</i>

	- The use case returns to step 2. A2 – Plan source limit reached: 3.1 If the user has reached the maximum number of sources allowed by their plan, the system displays: <i>"Source limit reached. Upgrade your plan."</i> - The use case ends. A3 – User deletes a source: 6.1 The user clicks the delete icon on an uploaded source. 6.2 The system removes the file from MinIO storage and deletes the record from the database. 6.3 The source is removed from the sources list.
--	--



Use Case ID	UC-04
Use Case Name	Submit Documents
Actor	User
Description	Allows an authenticated user to select source files and documents within a workspace and submit them for plagiarism analysis. The system validates the submission against plan limits and document type restrictions before queuing the analysis job.
Preconditions	The user is authenticated and has an active account.

	• A workspace exists and is open. • At least 2 source files have been uploaded to the workspace. • At least 1 document to check has been uploaded to the workspace.
Postconditions	• A submission record is created in the database with status pending. • The workspace status is updated to pending. • The analysis job is queued for asynchronous processing.
Includes	UC-04a – Analyze submission using AI model UC-04b – View analysis result
Main Flow	1. The user opens a workspace detail page. 2. The user reviews the uploaded source files and documents. 3. The user clicks submit for analysis. 4. The system checks that at least 2 sources and 1 document are present. 5. The system validates the document file types against allowed formats. 6. The system checks the user's plan to verify the monthly check limit has not been reached. 7. The system Creates a submission record and associates the selected sources and documents. 8. The system updates the workspace status to pending. 9. The system Queues the analysis job. (include: UC-04a) 10. The system returns the submission record to the frontend and displays processing status.
Alternative Flows	A1 – Insufficient sources: • 4.1 If fewer than 2 source files are present, the system displays: <i>"Minimum 2 sources required for analysis."</i> • 4.2 The use case ends. A2 – No documents uploaded: • 4.1 If no document to check is present, the system displays: <i>"At least 1 document to check is required."</i> • 4.2 The use case ends.

	A3 – Unsupported document type: • 5.1 If any document has an unsupported file extension, the system displays: <i>"Unsupported file type: [filename]."</i> • 5.2 The submission record is deleted. • 5.3 The use case ends. A4 – Plan limit reached: • 6.1 If the user has exhausted their monthly check quota, the system displays: <i>"Monthly limit reached. Upgrade your plan."</i> • 6.2 The submission record is deleted. • 6.3 The use case ends.
--	---

Use Case ID	UC-04a
Use Case Name	Analyze Submission Using AI Model
Actor	System (Celery worker), AI Microservice
Description	A background process triggered automatically upon submission. The Celery worker sends each submitted document along with presigned source file URLs to the AI microservice, which performs plagiarism detection and posts the result back to the system via a callback endpoint.
Preconditions	• A submission record exists with status pending. • All submitted files are stored in MinIO object storage. • The AI microservice is reachable.
Postconditions	• A DocumentResult record is created per document containing the plagiarism score, matched sources, and highlighted paragraphs. • The submission status is updated to completed. • The workspace status is updated to analyzed.
Included by	UC-04 – Submit Documents
Main Flow	1. Celery worker picks up the queued analysis job. 2. The system updates submission status to processing.

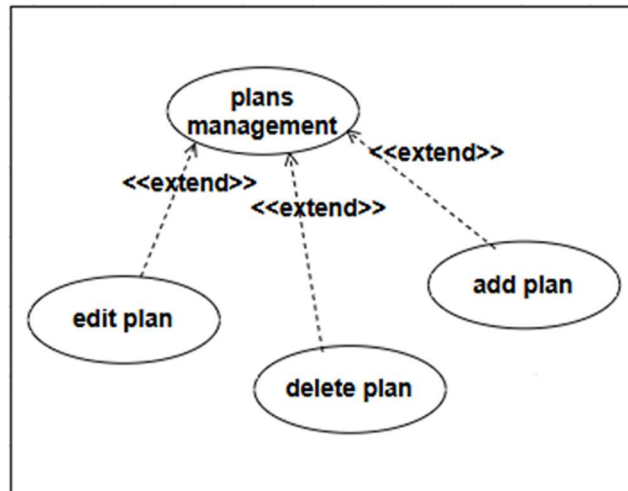
	3. The system retrieves presigned download URLs for all source files and documents from MinIO via the storage microservice. 4. The system sends a POST request to the AI microservice containing the document URL, source URLs, and a result callback URL. 5. AI Microservice downloads the document and source files from MinIO using the presigned URLs. 6. AI Microservice performs plagiarism detection and generates a plagiarism score, matched sources, and highlighted paragraphs. 7. AI Microservice posts the result to the system's callback endpoint. 8. The system receives the result and saves a DocumentResult record per document. 9. The system updates the submission status to completed and the workspace status to analyzed.
Alternative Flows	A1 - Analysis fails in AI microservice: • 6.1 If the AI microservice encounters an error during processing, it posts an error flag to the callback endpoint. • 6.2 The system sets the submission status to failed. • 6.3 The use case ends. A2 - Multiple documents in submission: • 4.1 Steps 4 through 8 are repeated for each document in the submission. • 4.2 The submission is marked completed only after all documents have been processed.
Exceptions	E1 – AI microservice unreachable: • If the POST request to the AI microservice fails, the Celery task retries up to 3 times with a 60-second delay. If all retries fail, the submission is marked as failed. E2 – Storage service unavailable: • If presigned URLs cannot be generated, the task fails and the submission is marked as failed.

Use Case ID	UC-04b
--------------------	--------

Use Case Name	View Analysis Result
Actor	User
Description	Allows an authenticated user to view the plagiarism analysis results for the latest submission in a workspace. Results include a per-document plagiarism score, original percentage, ranked matched sources with color-coded similarity bars, and highlighted plagiarized paragraphs.
Preconditions	- The user is authenticated and has an active account. - A completed submission exists for the workspace.
Postconditions	The analysis results are displayed to the user.
Included by	UC-04 – Submit Documents
Extended by	UC-04ba – Download Plagiarism Report
Main Flow	- The user opens a workspace detail page with a completed submission. - The system retrieves the latest completed submission and its document results from the database. - The system displays per-document result cards showing plagiarism score and original percentage. - The user clicks on a document result card to expand it. - The system displays matched sources with similarity percentages and color-coded bars. - The system displays highlighted plagiarized paragraphs with source attribution.
Alternative Flows	A1 – No completed submission exists: 2.1 If no completed submission exists for the workspace, the system displays: <i>"No submissions found. Please submit documents first."</i> 2.2 The use case ends. A2 – Analysis still in progress: 2.1 If the latest submission status is pending or processing, the system displays: <i>"Analysis in progress..."</i> and polls for updates. 2.2 Once the submission status changes to completed, the use case resumes from step 3.

	A3 – User views previous submission results: 1.1 The user clicks View Results on a previous submission from the submissions list. 1.2 The use case resumes from step 3 using the selected submission's results.
--	---

Use Case ID	UC-04ba
Use Case Name	Download Plagiarism Report
Actor	User
Description	Allows an authenticated user to download a PDF report of the analysis results for the current workspace. The report is generated client-side and includes plagiarism scores, matched sources, and highlighted paragraphs for all analyzed documents.
Preconditions	- The user is authenticated and has an active account. - A completed submission with results exists for the workspace.
Postconditions	A PDF report is generated and downloaded to the user's device.
Extends	UC-04b – View Analysis Result
Main Flow	- The user clicks the download PDF button on the results page. - The system collects the currently displayed result data from the frontend state. - The system generates a PDF document client-side using jsPDF containing workspace name, generation date, per-document scores, matched sources, and highlighted paragraphs.
Alternative Flows	A1 – PDF generation fails: 3.1 If the PDF library fails to generate the document, the system displays: <i>"PDF generation failed."</i> 3.2 The use case ends.



Use Case ID	UC-05
Use Case Name	Plans Management
Actor	Admin
Description	Allows an authenticated admin to manage the subscription plans available in the system. This is the base use case extended by more specific plan operations including adding, editing, and deleting plans.
Preconditions	The admin is authenticated and has an active account with role admin.
Postconditions	The requested plan operation is completed and reflected in the system.
Extended by	UC-05a, UC-05b, UC-05c
Main Flow	1. The admin navigates to the plans management page. 2. The system retrieves and displays all subscription plans including inactive ones. 3. The admin selects a plan operation (add, edit, or delete). 4. The system executes the selected operation via the appropriate extended use case.

Use Case ID	UC-05a
Use Case Name	Add Plan

Actor	Admin
Description	Allows an authenticated admin to create a new subscription plan by providing its name, price, monthly check limit, maximum sources, maximum documents, and allowed file formats.
Preconditions	The admin is authenticated and has an active account with role admin.
Postconditions	• A new subscription plan is created and saved in the database. • The new plan appears in the plans list and becomes available for users during registration.
Extends	UC-05 – Plans Management
Main Flow	1. The admin clicks the add plan button. 2. The system displays the ass plan modal with empty fields. 3. The admin enters plan name, price, checks per month, max sources, and max documents. 4. The admin clicks save. 5. The system validates the entered data. 6. The system creates the plan record and saves it to the database. 7. The system closes the modal and displays the new plan in the plans list.
Alternative Flows	A1 – Plan name is empty: • 5.1 If the plan name field is empty, the system displays: <i>"Name required."</i> • 5.2 The use case returns to step 3. A2 – Duplicate plan name: • 5.1 If a plan with the same name already exists, the system displays: <i>"A plan with this name already exists."</i> • 5.2 The use case returns to step 3. A3 – Invalid price: • 5.1 If the price is negative, the system displays: <i>"Price must be positive."</i> • 5.2 The use case returns to step 3.

Use Case ID	UC-05b
Use Case Name	Edit Plan
Actor	Admin
Description	Allows an authenticated admin to modify the details of an existing subscription plan including its name, price, monthly check limit, maximum sources, and maximum documents.
Preconditions	- The admin is authenticated and has an active account with role admin. - At least one plan exists in the system.
Postconditions	- The plan record is updated in the database. - The updated details are immediately reflected in the plans list and apply to new registrations.
Extends	UC-05 – Plans Management
Main Flow	- The admin clicks the edit icon on a plan card. - The system displays the edit plan modal pre-filled with the current plan details. - The admin modifies the desired fields and clicks save. - The system validates the updated data. - The system updates the plan record in the database. - The system closes the modal and reflects the updated details in the plans lists.
Alternative Flows	A1 – Plan name is empty: 5.1 If the plan name field is empty, the system displays: <i>"Name required."</i> 5.2 The use case returns to step 3. A2 – Invalid price: 5.1 If the price is negative, the system displays: <i>"Price must be positive."</i> 5.2 The use case returns to step 3.

Use Case ID	UC-05c
Use Case Name	Delete Plan
Actor	Admin
Description	Allows an authenticated admin to permanently delete an existing subscription plan from the system. Deleted plans are no longer available for new user registrations.
Preconditions	• The admin is authenticated and has an active account with role admin. • At least one plan exists in the system.
Postconditions	• The plan record is permanently deleted from the database. • The plan no longer appears in the plans list or the registration plan selection page.
Extends	UC-05 – Plans Management
Main Flow	1. The admin clicks the delete icon on a plan card. 2. The system displays a confirmation modal warning that the plan will be permanently deleted and affected user will need to be reassigned. 3. The admin clicks delete to confirm. 4. The system deletes the plan record from the database. 5. The system closes the modal and removes the plan from the plans list.

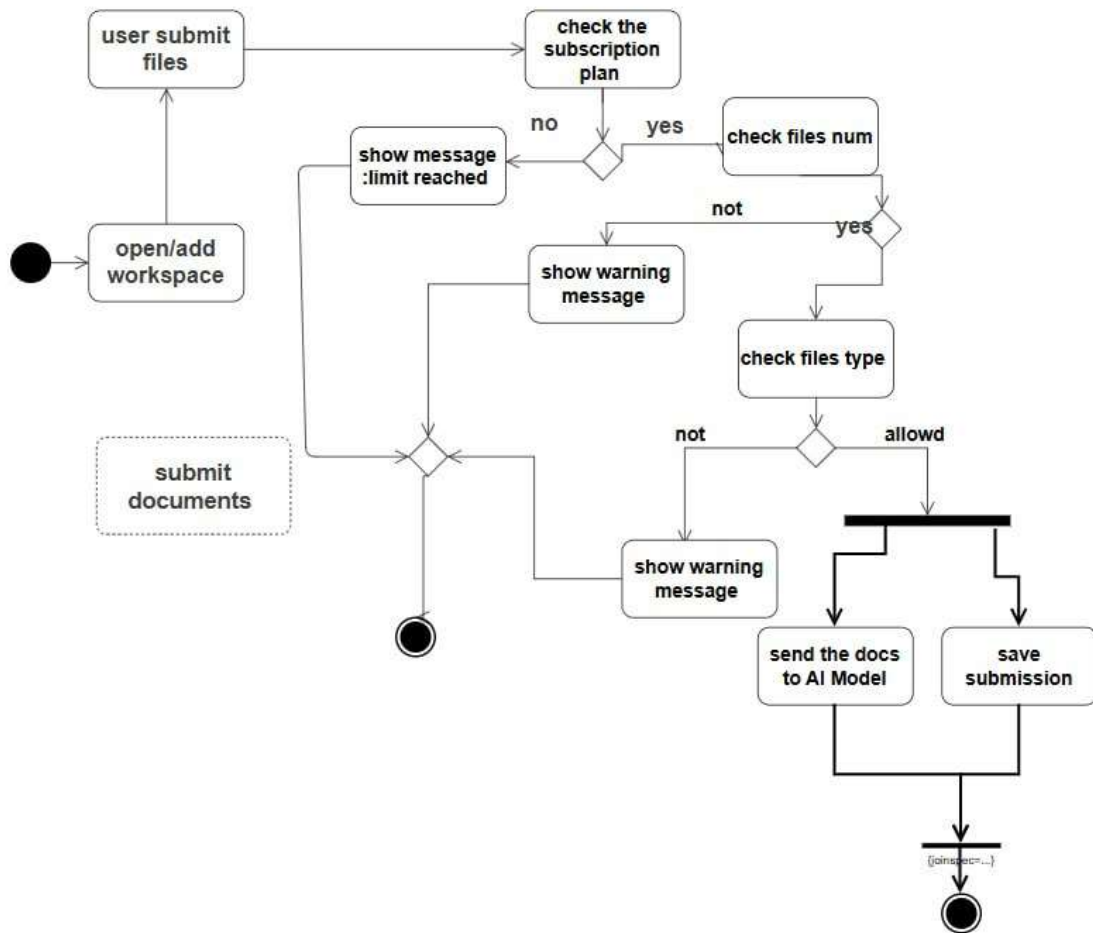
Use Case ID	UC-06
Use Case Name	Manage Account
Actor	Admin
Description	Allows an authenticated admin to view all registered user accounts in the system and update their status. The admin can deactivate accounts to block user access or reactivate them to restore it.
Preconditions	The admin is authenticated and has an active account with role admin.

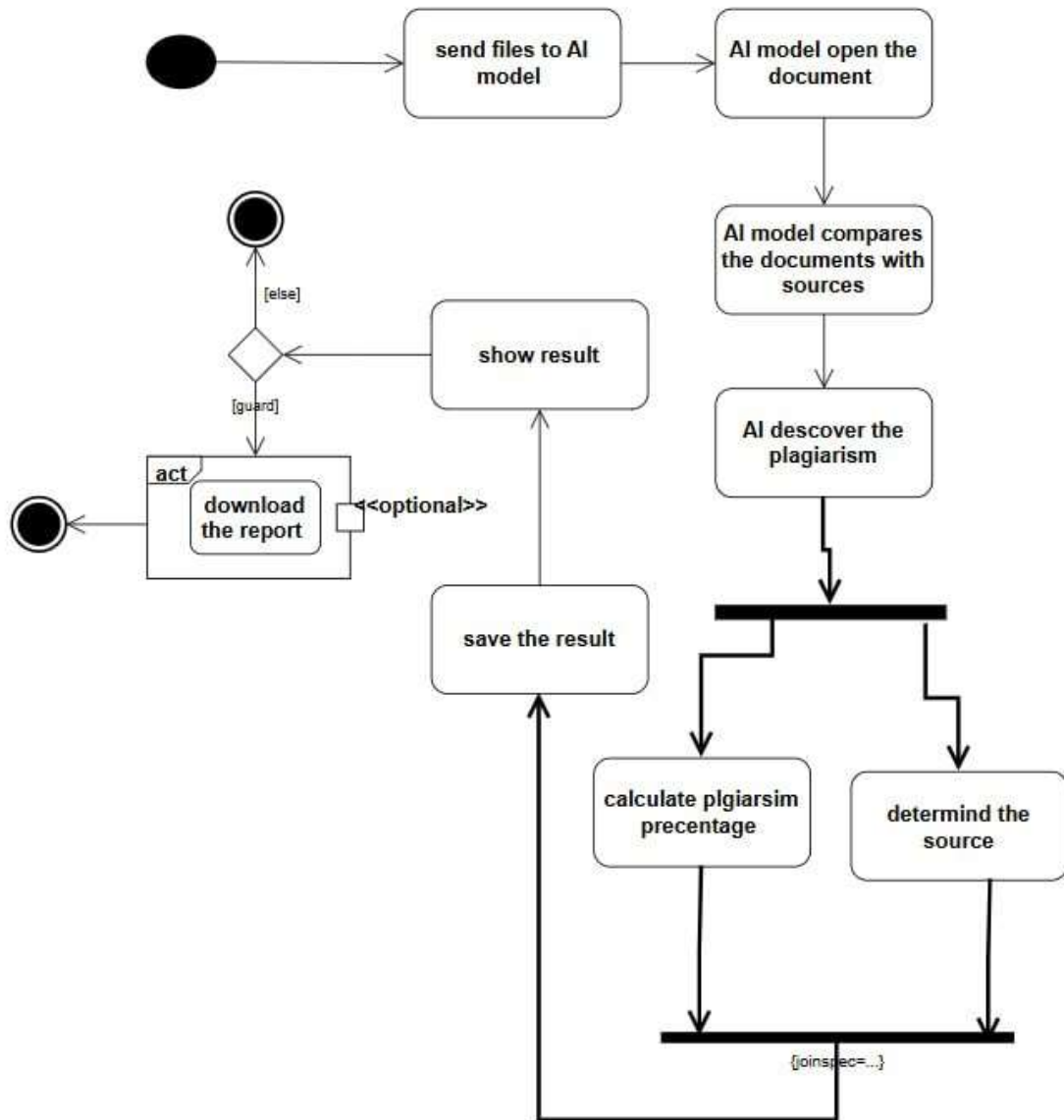
Postconditions	The affected user's access is immediately updated based on their new status.
Main Flow	1. The admin navigates to the accounts management page. 2. The system retrieves and displays all registered user accounts with name, email, role, plan, status, and join data. 3. The admin clicks the edit icon on a user account. 4. The system displays the edit account modal showing the user's details and current status. 5. The admin selects the new status and clicks save changes. 6. The system updates the account status in the database. 7. The system closes the modal and reflects the updated status in the accounts list.
Alternative Flows	A1 - Admin deactivates an account: • 5.1 Admin sets status to Inactive. • 5.2 System updates the account status to inactive and sets is_active to false. • 5.3 The affected user is blocked from logging in and any active session is invalidated. • 5.4 The use case resumes from step 8. A2 – Admin reactivates an account: • 5.1 Admin sets status to Active. • 5.2 System updates the account status to active and sets is_active to true. • 5.3 The affected user can log in again. • 5.4 The use case resumes from step 8.

Use Case ID	UC-07
Use Case Name	View Submission History
Actor	User

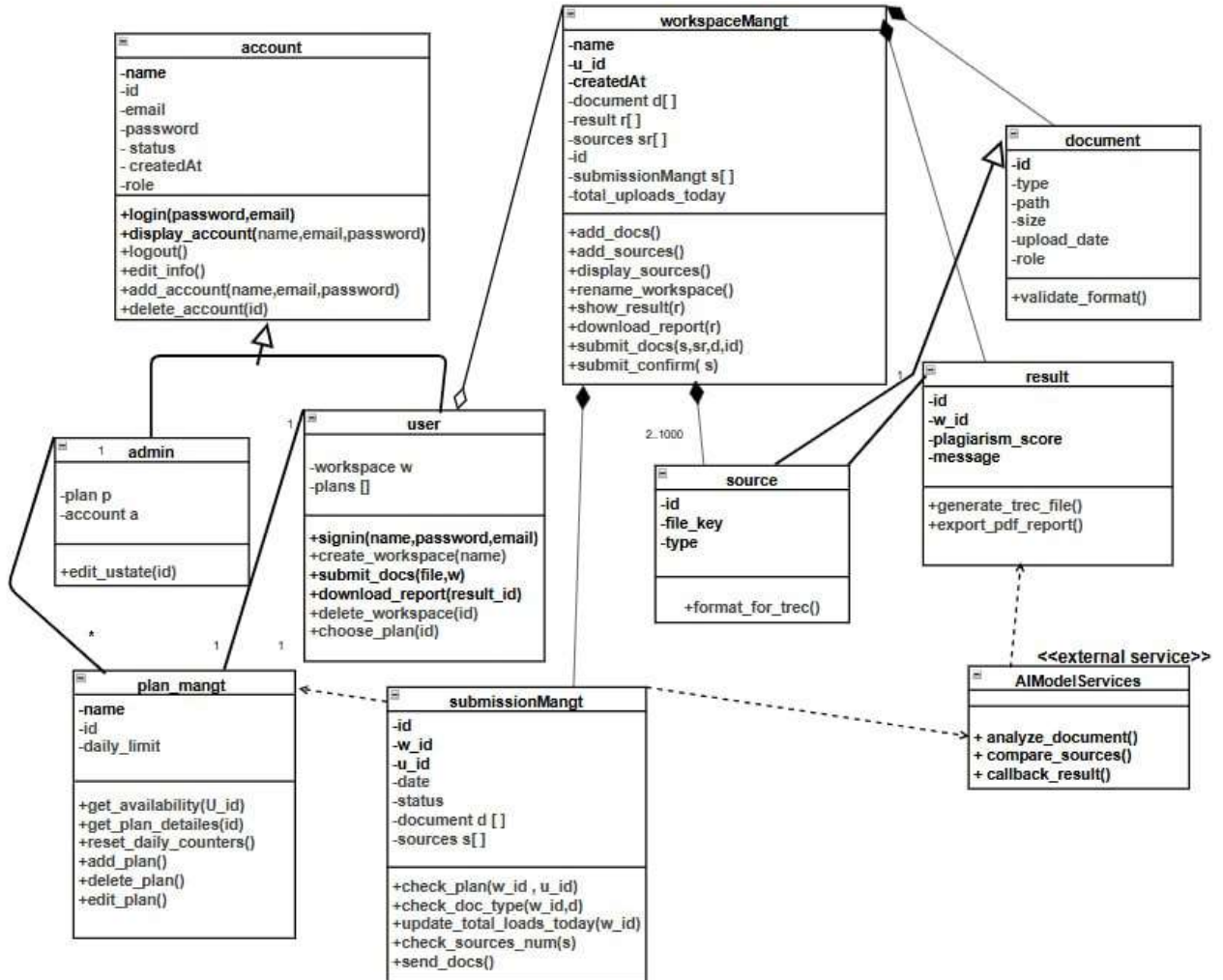
Description	Allows an authenticated user to view a chronological list of all their past submissions across all workspaces. The user can expand any completed submission to inspect per-document plagiarism results including matched sources and highlighted plagiarized paragraphs.
Preconditions	The user is authenticated and has an active account.
Postconditions	Submission history is displayed to the user.
Main Flow	1. The user navigates to the history page. 2. The system retrieves all submissions belonging to the authenticated user ordered by date descending. 3. The system displays each submission with workspace name, submission date, worst plagiarism score, and status.
Alternative Flows	A1 – User expands a completed submission: • 3.1 User clicks the View button on a completed submission entry. • 3.2 System expands the submission row and displays per-document result cards. • 3.3 Each card shows the document name, plagiarism score, original percentage, and matched sources with color-coded similarity bars.

5.2 Analysis Activity Diagram

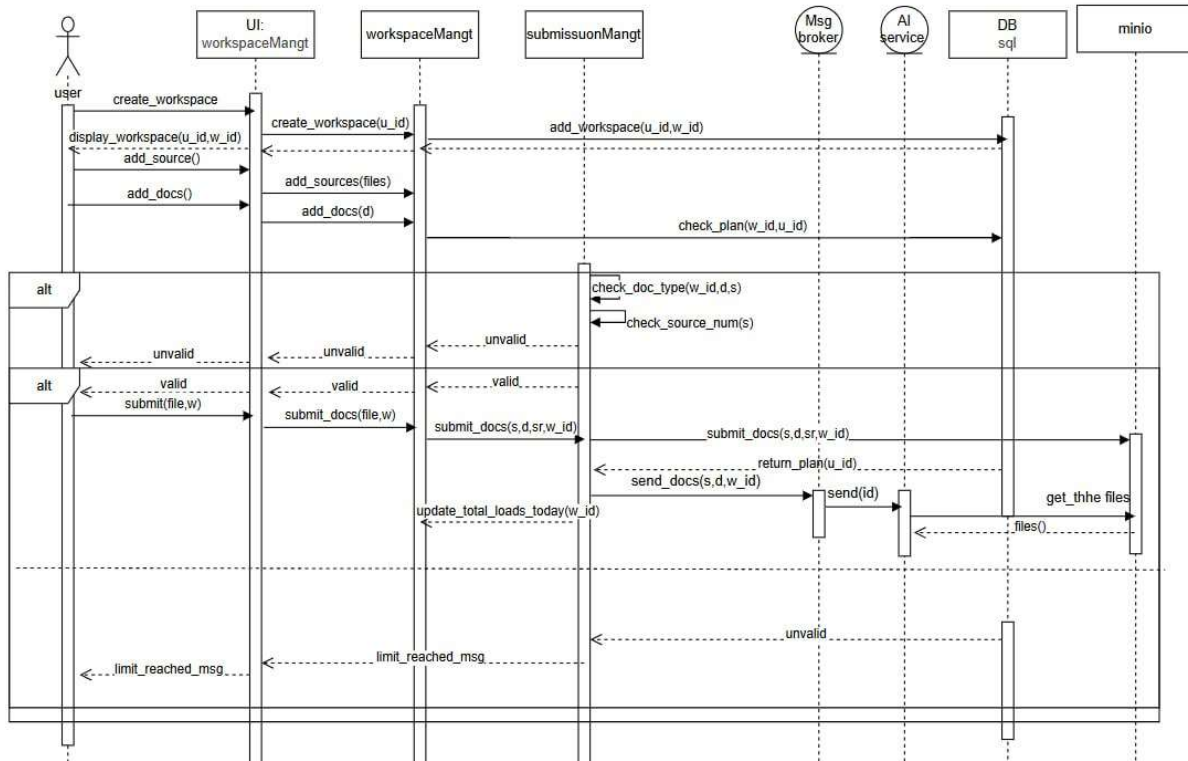


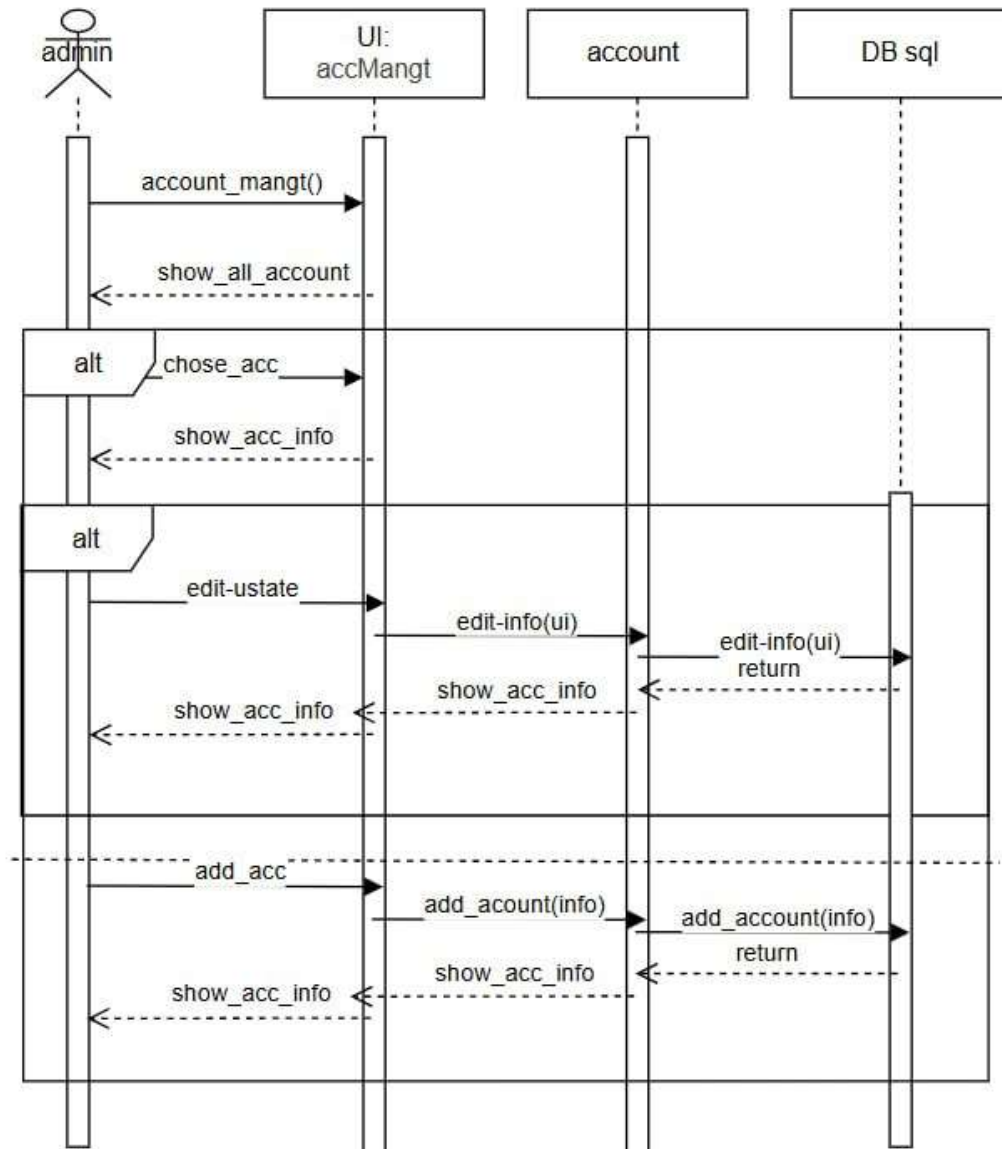


5.3 Class Diagram



5.4 Sequence Diagram





6. Initial Test Cases

Table 1 Test cases for User Login (UC-2)

Test case id	Test case title	Test steps	Expected result
Tc-01	Check results on login with valid credentials	1. Launch the website. 2. Navigate to the Login page. 3. Enter valid email and password. 4. Click the Login button.	User is authenticated and redirected to the Dashboard.
Tc-02	Check results on login with invalid password	1. Launch the website. 2. Navigate to the Login page. 3. Enter a valid email and an incorrect password. 4. Click the Login button.	Error message: Invalid email or password.
Tc-03	Check results on login with non-existent email	1. Launch the website. 2. Navigate to the Login page. 3. Enter an unregistered email address. 4. Click the Login button.	Error message: Invalid email or password.
Tc-04	Check results on login with empty fields	1. Launch the website. 2. Navigate to the Login page. 3. Leave email and password fields empty. 4. Click the Login button.	Error message: required fields must be filled.
Tc-05	Check results on login with inactive account	1. Launch the website. 2. Navigate to the Login page. 3. Enter credentials of a deactivated user. 4. Click the Login button.	Error message: Your account has been deactivated.

Table 2 Test cases for User Registration (UC-1)

Test case id	Test case title	Test steps	Expected result
Tc-06	Check results on successful registration with OTP verification	1. Launch the website. 2. Navigate to the Registration page. 3. Enter valid name, email, password and confirm password. 4. Select a subscription plan. 5. Click Send Verification Code. 6. Enter the correct 6-digit OTP received by email. 7. Click Verify & Create Account.	Account created. User is auto-logged in and redirected to the Dashboard.
Tc-07	Check results on registration with already existing email	1. Launch the website. 2. Navigate to the Registration page. 3. Enter an email already registered in the system. 4. Fill remaining fields and click Send Verification Code.	Error message: An account with this email already exists.

Tc-08	Check results on registration with mismatched passwords	1. Launch the website. 2. Navigate to the Registration page. 3. Enter a valid email. 4. Enter different values in password and confirm password fields. 5. Click Continue.	Error message: Passwords do not match.
Tc-09	Check results on registration with invalid OTP	1. Launch the website. 2. Complete registration form and select a plan. 3. Click Send Verification Code. 4. Enter an incorrect or expired OTP. 5. Click Verify & Create Account.	Error message: Invalid or expired verification code.
Tc-10	Check results on OTP resend	1. Launch the website. 2. Complete registration up to OTP step. 3. Wait for resend timer to expire. 4. Click Resend code. 5. Check email for new OTP.	New 6-digit OTP sent. Previous OTP is invalidated.

Table 3 Test cases for Workspace Management (UC-3)

Test case id	Test case title	Test steps	Expected result
Tc-11	Check results on creating a new workspace	1. Log in as a regular user. 2. Navigate to the Workspaces page. 3. Click New Workspace. 4. Enter a workspace name. 5. Click Create.	New workspace is created and appears in the list.
Tc-12	Check results on creating a workspace with empty name	1. Log in as a regular user. 2. Click New Workspace. 3. Leave the name field empty. 4. Click Create.	Error message: Name is required.
Tc-13	Check results on renaming a workspace	1. Log in as a regular user. 2. Navigate to Workspaces. 3. Click the Edit (pencil) icon on a workspace. 4. Enter a new name. 5. Click Save.	Workspace is renamed successfully.
Tc-14	Check results on deleting a workspace	1. Log in as a regular user. 2. Navigate to Workspaces. 3. Click the Delete (trash) icon on a workspace. 4. Confirm deletion in the modal.	Workspace is permanently deleted and removed from the list.
Tc-15	Check results on uploading a source file to a workspace	1. Log in and open a workspace. 2. In the Sources panel, click Add Sources or drag a file. 3. Select a valid .pdf, .docx, or .txt file. 4. Confirm upload.	Source file appears in the sources list with name and size.

Tc-16	Check results on uploading a source file with unsupported format	1. Log in and open a workspace. 2. Attempt to upload a file with an unsupported extension (e.g. .mp4). 3. Confirm upload.	Error message: Unsupported file type.
--------------	--	---	---------------------------------------

Table 4 Test cases for Submit Documents (UC-04)

Test case id	Test case title	Test steps	Expected result
Tc-17	Check results on submitting documents for analysis with valid inputs	1. Log in as a regular user. 2. Open a workspace. 3. Upload at least 2 source files. 4. Upload at least 1 document to check. 5. Click Submit for Analysis.	Submission accepted. Workspace status changes to Pending. Processing indicator displayed.
Tc-18	Check results on submitting with fewer than 2 source files	1. Log in as a regular user. 2. Open a workspace. 3. Upload only 1 source file. 4. Upload 1 document to check. 5. Click Submit for Analysis.	Error message: Minimum 2 sources required for analysis.
Tc-19	Check results on submitting with no documents to check	1. Log in as a regular user. 2. Open a workspace. 3. Upload 2 source files. 4. Do not upload any document to check. 5. Click Submit for Analysis.	Error message: At least 1 document to check is required.
Tc-20	Check results on submitting a document with unsupported file format	1. Log in as a regular user. 2. Open a workspace. 3. Upload 2 source files. 4. Upload a document with an unsupported extension (e.g. .mp4). 5. Click Submit for Analysis.	Error message: Unsupported file type: [filename].
Tc-21	Check results on submitting when monthly check limit is reached	1. Log in as a user who has exhausted their monthly check quota. 2. Open a workspace with at least 2 sources and 1 document. 3. Click Submit for Analysis.	Error message: Monthly limit reached. Upgrade your plan.
Tc-22	Check results on uploading a document to check	1. Log in and open a workspace. 2. In the Documents panel, click Add Documents or drag a file. 3. Select a valid .pdf, .docx, or .txt file.	Document appears in the documents list with name and size.

Table 5 Test cases for View Analysis Results (UC-04b)

Test case id	Test case title	Test steps	Expected result
Tc-23	Check results on viewing analysis results after submission completes	1. Log in and open a workspace. 2. Submit documents for analysis. 3. Wait for analysis to complete. 4. View the results panel.	Per-document plagiarism score, original percentage, and matched sources displayed.

Tc-24	Check results on expanding a document result card	1. Log in and open a workspace with a completed submission. 2. Click on a document result card to expand it.	Matched sources with color-coded similarity bars and percentages displayed.
Tc-25	Check results on viewing highlighted plagiarized paragraphs	1. Log in and open a workspace with a completed submission. 2. Expand a document result card. 3. Scroll to the plagiarised paragraphs section.	Highlighted paragraphs displayed with source attribution and match percentage.
Tc-26	Check results on viewing results when analysis is still in progress	1. Log in and open a workspace. 2. Submit documents. 3. Navigate back to workspace before analysis completes.	Message displayed: Analysis in progress... System polls for updates automatically.
Tc-27	Check results on downloading a PDF report	1. Log in and open a workspace with completed results. 2. Click Download PDF. 3. Wait for the report to generate.	PDF report downloaded to the user's device containing scores, matched sources, and highlighted paragraphs.

Table 6 Test cases for Submission History (UC-07)

Test case id	Test case title	Test steps	Expected result
Tc-28	Check results on viewing submission history	1. Log in as a regular user. 2. Navigate to the History page.	All past submissions displayed ordered by date descending with workspace name, date, worst score, and status.
Tc-29	Check results on expanding a completed submission in history	1. Log in and navigate to the History page. 2. Click View on a completed submission entry.	Submission row expands showing per-document result cards with matched sources and similarity bars.
Tc-30	Check results on viewing history with no past submissions	1. Log in as a new user with no submissions. 2. Navigate to the History page.	Message displayed: No submissions yet. Submit documents in a workspace to see results here.
Tc-31	Check results on viewing a submission that is still processing in history	1. Log in and navigate to the History page. 2. Locate a submission with status Pending or Processing.	Submission displayed with current status only. View button not available.

Table 7 Test cases for Plans Management by Admin (UC-05)

Test case id	Test case title	Test steps	Expected result
Tc-32	Check results on viewing all plans as Admin	1. Log in as Admin. 2. Navigate to the Plans Management page.	All subscription plans displayed including inactive ones with name, price, and limits.
Tc-33	Check results on adding a new plan	1. Log in as Admin. 2. Navigate to Plans Management. 3. Click Add Plan. 4. Enter plan name, price, checks per month, max sources, max documents.	New plan created and displayed in the plans list. Plan available for new user registrations.

		5. Click Save.	
Tc-34	Check results on adding a plan with empty name	1. Log in as Admin. 2. Navigate to Plans Management. 3. Click Add Plan. 4. Leave the plan name field empty. 5. Click Save.	Error message: Name required.
Tc-35	Check results on adding a plan with negative price	1. Log in as Admin. 2. Navigate to Plans Management. 3. Click Add Plan. 4. Enter a negative value in the price field. 5. Click Save.	Error message: Price must be positive.
Tc-36	Check results on editing an existing plan	1. Log in as Admin. 2. Navigate to Plans Management. 3. Click the Edit icon on a plan. 4. Modify the price or checks per month. 5. Click Save.	Plan updated successfully. Updated details reflected in the plans list.
Tc-37	Check results on deleting a plan	1. Log in as Admin. 2. Navigate to Plans Management. 3. Click the Delete icon on a plan. 4. Confirm deletion in the modal.	Plan permanently deleted and removed from the plans list and the registration plan selection page.

Table 8 Test cases for Manage Account by Admin (UC-06)

Test case id	Test case title	Test steps	Expected result
Tc-38	Check results on viewing all user accounts as Admin	1. Log in as Admin. 2. Navigate to the Accounts Management page.	All registered user accounts displayed with name, email, role, plan, status, and join date.
Tc-39	Check results on deactivating a user account	1. Log in as Admin. 2. Navigate to Accounts Management. 3. Click the Edit icon on an active user account. 4. Set status to Inactive. 5. Click Save Changes.	Account status updated to Inactive. User is blocked from logging in.
Tc-40	Check results on reactivating a user account	1. Log in as Admin. 2. Navigate to Accounts Management. 3. Click the Edit icon on an inactive user account. 4. Set status to Active. 5. Click Save Changes.	Account status updated to Active. User can log in again.
Tc-41	Check results on cancelling account status edit	1. Log in as Admin. 2. Navigate to Accounts Management. 3. Click the Edit icon on a user account. 4. Click Cancel without making changes.	Modal closed. Account status remains unchanged.

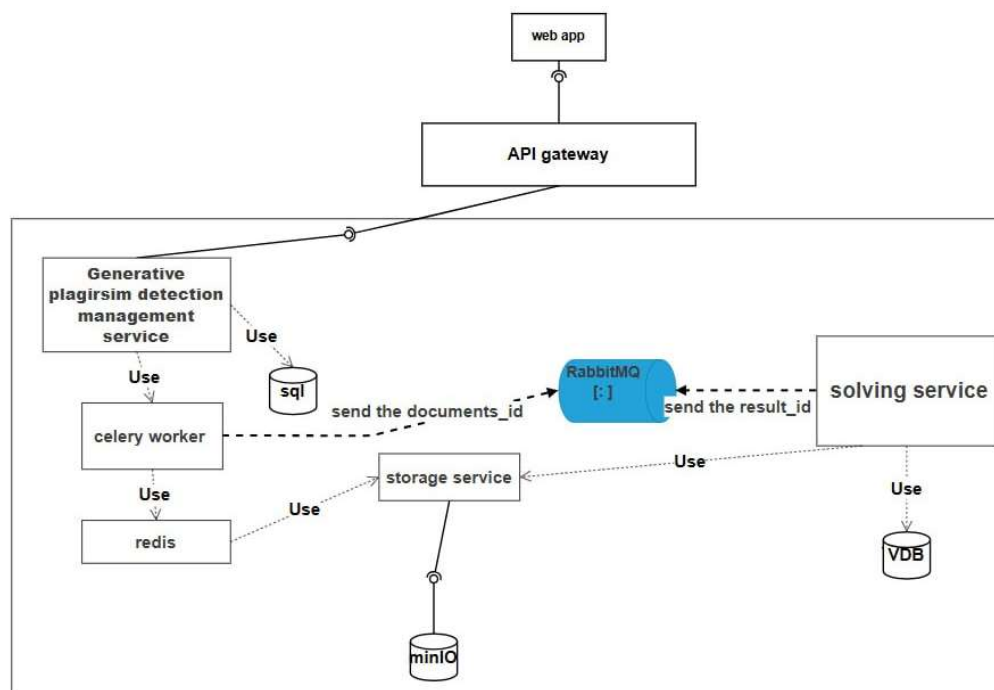
Chapter 5: System Design

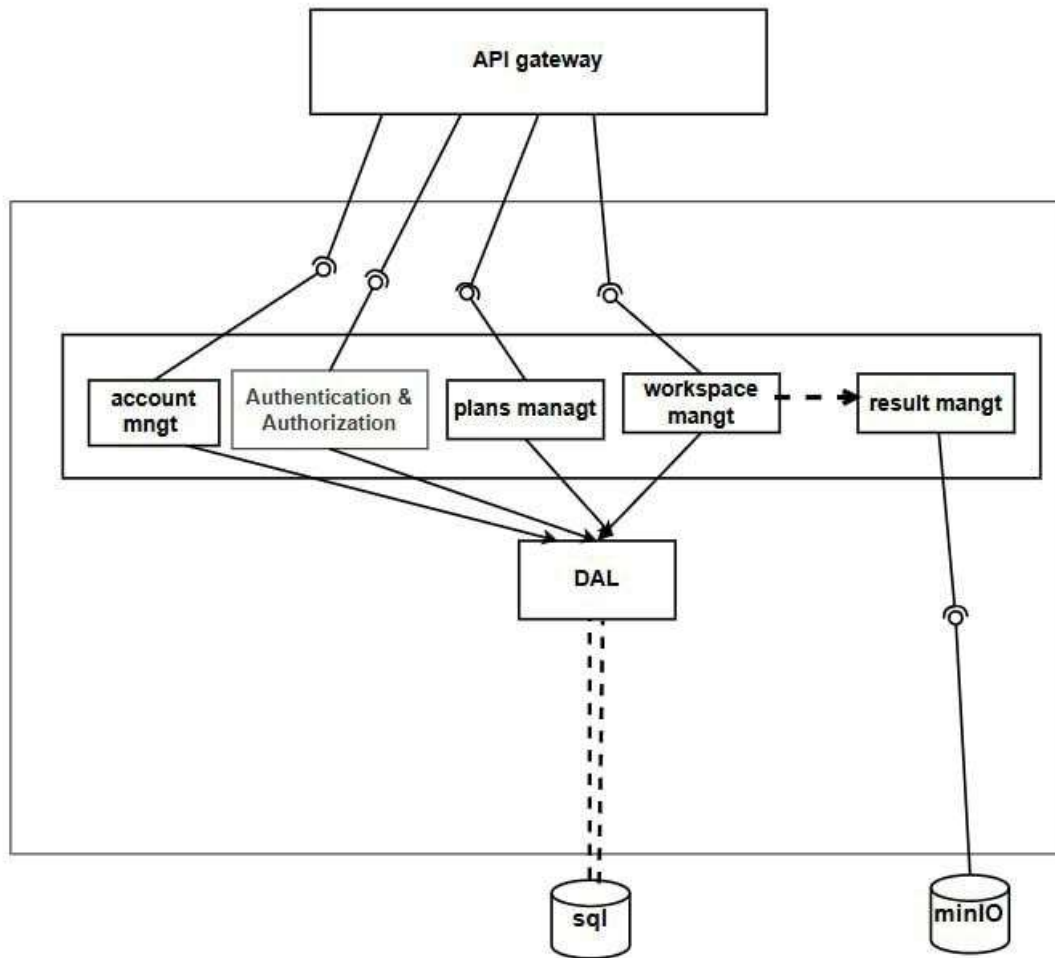
5.1 Introduction

This chapter offers a comprehensive overview of the design phase of the Generative Plagiarism Detection system, encompassing the system architecture, component design, and the interactions between the system's core components. It underscores the importance of these design decisions in building a robust and modular system that cleanly separates user-facing functionality from the underlying AI detection logic, ensuring each component can be developed and maintained independently.

5.2 System Architecture

In this section we will discuss the system design of the Generative Plagiarism Detection platform, where system design involves defining the overall structure and architecture of the software, ensuring that all components work seamlessly together. This phase establishes the blueprint for efficient data flow, interaction, and integration between the web application, the storage service, and the AI analysis microservice.





5.2.1 Component Functionalities

1. API Gateway: This component is responsible for routing all external API calls from the web application to their respective service endpoints, ensuring secure and efficient communication between client requests and the server-side logic. It serves as the single-entry point for all incoming requests, handling authentication token verification before forwarding requests to the appropriate internal service.

2. Generative Plagiarism Detection Management Service: This is the core backend service of the system. It processes incoming requests from the API gateway and coordinates all business logic across the system's functional modules including account management, plan

management, workspace management, and result management. It invokes the appropriate module based on the request and returns the response to the client.

3. Account Management: Handles all user and admin account-related functionalities including registration, OTP email verification, login, profile updates, password changes, account deletion, and admin-level account status management.

4. Authentication and Authorization: Responsible for verifying user identity and enforcing role-based access control across all endpoints. It issues and validates JWT access and refresh tokens, ensures that protected routes are only accessible to authenticated users, and restricts admin-only operations to users with the admin role.

5. Plans Management: Handles all subscription plan-related operations including plan creation, editing, and deletion by the admin, as well as plan retrieval for the registration plan selection page. It enforces plan-based quotas such as monthly check limits, maximum sources, and maximum documents per workspace.

6. Workspace Management: Manages all workspace-related operations including workspace creation, renaming, and deletion. It handles the upload and storage of source reference files and suspicious documents, enforcing plan limits and file format restrictions before delegating file storage to the storage service.

7. Celery Worker: Responsible for executing long-running analysis jobs asynchronously in the background. When a submission is queued, the Celery worker picks up the task from the message broker, retrieves presigned file URLs from the storage service, and dispatches the analysis request to the solving service. This ensures that document analysis does not block the main web application.

8. RabbitMQ: Acts as the message broker between the management service and the solving service. It receives analysis job messages from the Celery worker containing document identifiers and relays result notifications back from the solving service upon completion, enabling fully decoupled asynchronous communication between the two services.

9. Storage Service: Manages all file storage operations for the system. It handles uploading source files and documents to object storage, generating presigned download URLs for the Celery worker and the solving service to access files directly, and deleting files when sources or documents are removed by the user.

10. Object Storage (MinIO): Stores all uploaded files including source reference documents and suspicious documents submitted for analysis. Files are stored as binary objects identified by a unique file key, which is the only reference persisted in the SQL database.

11. Solving Service (AI Microservice): Responsible for the plagiarism detection logic. It receives analysis requests containing presigned document and source URLs, downloads the files, encodes them into dense vector representations, performs semantic retrieval against the vector database to identify the most similar source documents, calculates plagiarism scores, and posts the results back to the management service via a callback endpoint.

12. Vector Database (VDB): Stores the dense vector embeddings of encoded source document chunks generated by the solving service. It enables fast semantic similarity search during the retrieval phase, allowing the solving service to efficiently identify which source documents are most semantically similar to a given suspicious document.

13. DAL (Data Access Layer): Acts as the intermediary between the business logic modules and the SQL database. It provides a consistent interface for fetching, storing, updating, and

deleting structured data including user accounts, plans, workspaces, submissions, and analysis results.

14. SQL Database: Stores all structured persistent data for the system including user accounts, subscription plans, workspace records, uploaded file metadata, submission records, and analysis results. It serves as the single source of truth for all relational data within the management service.

5.2.2 System Architecture

The Generative Plagiarism Detection system is built on a microservices architecture, where the system is decomposed into independently deployable services that communicate over well-defined HTTP APIs, allowing each component to be developed, tested, and updated in isolation. Within each service, a layered architecture is applied to maintain a clean separation of responsibilities.

1. Client Side: The topmost layer where the user interacts with the system through a web-based interface, handling all user-facing workflows including registration, workspace management, document submission, and result visualization. All communication with the backend is routed through the API gateway, which serves as the single-entry point for all client requests, handling authentication verification and request routing.

2. Server-Side Layers:

- **Business Logic Layer:** Contains the core functionality of the system, distributed across two independent microservices. The Management Service handles account management, authentication and authorization, plan management, workspace management, and result management. The Solving Service is dedicated to plagiarism detection logic including document encoding, semantic retrieval, and score calculation. The Celery worker operates within this layer, handling asynchronous task

dispatch between the two services via the message broker.

- **Data Access Layer:** Provides an abstraction for interacting with the database, implemented within the management service. All business logic modules interact with the database exclusively through this layer.

3. Storage Layer: An independent microservice responsible for managing all file storage operations. Uploaded files are stored in object storage and accessed via presigned URLs, ensuring raw file data never passes through the main application server.

4. Database Layer: The system uses two types of persistent storage. The SQL database stores all structured relational data and is accessed exclusively through the data access layer. The vector database stores dense vector embeddings of document chunks and is accessed directly by the solving service during semantic similarity search.

5.2.3 Design Patterns in Use

1. Repository Pattern:

- **Description:** The Data Access Layer within the management service implements the Repository pattern, abstracting all database operations behind a consistent interface. Business logic modules interact with the database exclusively through this layer without any direct database calls.
- **Purpose:** Decouples business logic from data persistence, promoting testability and flexibility. It allows the underlying database to be switched or modified without affecting the rest of the system.

2. State Pattern:

- **Description:** The submission workflow in the system involves clearly defined state transitions. A submission progresses through states — Pending, Processing, Completed, and Failed — and a workspace progresses through Draft, Pending, and Analyzed. The system's behavior changes based on the current state, for example blocking result display until the submission reaches the Completed state.
- **Purpose:** Encapsulates state-specific behavior and allows the system to respond dynamically to state changes, ensuring that each state enforces its own rules and transitions correctly.

3. Observer Pattern:

- **Description:** The asynchronous communication between the management service and the solving service follows an observer-like pattern through the message broker. When a submission is queued, the Celery worker dispatches the task and the solving service processes it independently, posting the result back to the management service via a callback endpoint upon completion.
- **Purpose:** Decouples the submission workflow from the analysis process, ensuring that the management service does not need to actively wait for or poll the solving service.

4. Dependency Injection Pattern:

- **Description:** Services and components within the system depend on each other through injected interfaces rather than hardcoded dependencies. For example, the data access layer, storage client, and task queue are configured and injected into the business logic modules through the application's settings and dependency management.
- **Purpose:** Improves testability and flexibility by making dependencies explicit and replaceable, allowing individual components to be tested or swapped in isolation without modifying dependent code.

5.3 Detailed design for system component

In this section we dive into the detailed design for each component of the system. For each component, we present its class diagram illustrating the relationships and responsibilities between its classes, along with the design patterns applied.

5.3.1 Design Class Diagrams

1. User Authentication and Account Management

This class diagram illustrates the relationships and responsibilities between the components involved in user authentication and account management. It includes the following primary entities:

1. **UserRepository:** Handles all database operations related to user accounts, including fetching, creating, updating, and deleting user records.
2. **AuthService:** Provides authentication services including credential validation, JWT token generation, OTP creation and verification, and password hashing.
3. **BaseAuthView:** A base class for all views that require authentication services, providing shared access to the AuthService.
4. **LoginView, RegisterView, LogoutView:** Represent the endpoints for user login, registration with OTP verification, and logout respectively.
5. **ProfileView:** Represents the endpoint for retrieving and updating user profile information including name, email, and password.
6. **AdminAccountView:** Represents the endpoint for admin-level account management, allowing the admin to list all user accounts and update their

status.

Relationships:

1. Dependency:

- AuthService depends on UserRepository for all user data operations.
- BaseAuthView depends on AuthService for authentication and authorization services.

2. Inheritance:

- LoginView, RegisterView, LogoutView, ProfileView, and AdminAccountView all inherit from BaseAuthView.

2. Workspace Management

This class diagram illustrates the relationships and responsibilities between the components involved in workspace and file management. It includes the following primary entities:

1. **WorkspaceRepository:** Handles all database operations related to workspaces, sources, and documents.
2. **StorageService:** Handles file upload, retrieval, and deletion operations by communicating with the storage microservice.
3. **WorkspaceService:** Provides the core business logic for workspace operations, coordinating between the repository and the storage service.

4. **WorkspaceView:** Represents the endpoints for creating, listing, renaming, and deleting workspaces.
5. **SourceView:** Represents the endpoints for uploading and deleting source reference files.
6. **DocumentView:** Represents the endpoints for uploading and deleting suspicious documents.

Relationships:

1. Dependency:

- WorkspaceService depends on WorkspaceRepository for all data operations.
- WorkspaceService depends on StorageService for all file storage operations.

2. Inheritance:

- WorkspaceView, SourceView, and DocumentView all inherit from a common BaseWorkspaceView that provides shared access to the WorkspaceService.

3. Submission and Analysis

This class diagram illustrates the relationships and responsibilities between the components involved in document submission and asynchronous analysis. It includes the following primary entities:

1. **SubmissionRepository:** Handles all database operations related to submissions and their associated sources and documents.
2. **ResultRepository:** Handles all database operations related to analysis results, matched sources, and highlighted paragraphs.
3. **SubmissionService:** Provides the core business logic for submission validation including plan limit checks, document type checks, and task queuing.
4. **AnalysisTask:** Represents the Celery background task responsible for dispatching analysis requests to the solving service and processing the results upon callback.
5. **SubmitView:** Represents the endpoint for submitting a workspace for analysis.
6. **ResultView:** Represents the endpoint for retrieving analysis results for a workspace.
7. **CallbackView:** Represents the internal callback endpoint that receives analysis results posted by the solving service.

Relationships:

1. **Dependency:**
 - SubmissionService depends on SubmissionRepository for submission data operations.
 - SubmissionService depends on ResultRepository for result storage operations.

- AnalysisTask depends on SubmissionService to retrieve file URLs and dispatch requests.

2. Inheritance:

- SubmitView, ResultView, and CallbackView inherit from a common BaseView providing shared authentication and permission handling.

4. Plans Management

This class diagram illustrates the relationships and responsibilities between the components involved in subscription plan management. It includes the following primary entities:

1. **PlanRepository:** Handles all database operations related to subscription plans.
2. **PlanService:** Provides the core business logic for plan operations including creation, update, deactivation, and quota enforcement.
3. **PlanView:** Represents the endpoints for listing, creating, editing, and deleting subscription plans.

Relationships:

1. Dependency:

- PlanService depends on PlanRepository for all plan data operations.

- PlanView depends on PlanService for all business logic operations.

2. Access Control:

- Read operations on PlanView are publicly accessible to support the registration plan selection page.
- Write operations on PlanView are restricted to users with the admin role.

Design Pattern: State Pattern

The State Pattern is applied in the system to manage the lifecycle of submissions and workspaces, both of which progress through clearly defined states during the analysis workflow.

The diagram consists of the following key components:

1. Interface: SubmissionState This defines a common interface with a method `nextState(submission) : void` that must be implemented by all concrete states. Each state class implements this interface to define its own behavior and transition logic.

2. Concrete States (PendingState, ProcessingState, CompletedState, FailedState) Each of these classes implements the SubmissionState interface and represents a distinct phase in the submission lifecycle. A submission begins in PendingState when it is first created, transitions to ProcessingState when the Celery worker picks up the analysis task, moves to CompletedState when the solving service successfully posts the result back, and transitions to FailedState if an error occurs at any point during processing.

3. Submission Class This class holds a reference to a SubmissionState object representing the current state of the submission. It provides a `changeState(state) : void` method that

allows the system to transition the submission to a different state as the analysis progresses.

4. WorkflowManager Class This class manages state transitions across the submission lifecycle. It maintains a state map tracking available states and their corresponding classes, and provides methods to retrieve the next valid state and enforce transition rules based on the current submission status.

How the State Pattern Works in This Design:

1. **Initial State:** A submission is created in PendingState when the user clicks Submit for Analysis.
2. **State Transitions:**
 - The submission transitions to ProcessingState when the Celery worker picks up the task.
 - It transitions to CompletedState when the solving service posts the analysis result via the callback endpoint.
 - It transitions to FailedState if the solving service reports an error or all retry attempts are exhausted.
3. **Workspace State:**
 - In parallel, the workspace transitions from Draft to Pending when a submission is queued, and from Pending to Analyzed when the submission reaches CompletedState, reflecting the overall analysis status to the user.

Chapter 6: AI Service Experiments and Evaluation

6.1 Introduction

This chapter presents the source retrieval approach developed for the AI analysis microservice of the Generative Plagiarism Detection system, along with the experiments conducted to evaluate and improve its performance. Since the PAN 2026 task dataset is not publicly released, a proxy evaluation dataset was constructed from an existing benchmark to enable systematic experimentation. The chapter covers the proxy dataset construction process, the experimental pipeline designs, the evaluation results on both the proxy dataset and the official PAN 2026 test dataset, and a discussion of the findings. Additional experiments are ongoing and their results will be incorporated in the final version of this chapter.

6.2 Proxy Evaluation Dataset

Since the official PAN 2026 dataset was not available during development, a proxy evaluation dataset was constructed from the PAN 2025 Text Alignment validation set, which contains (suspicious document, source document) pairs with character-level XML plagiarism annotations marking exact plagiarized spans on both sides of each pair.

6.2.1 Dataset Construction Process

Step 1 – Sampling: 100 suspicious documents were randomly sampled along with their aligned source documents.

Step 2 — Source Chunking: Each source document was chunked into paragraph-level chunks by splitting on blank lines and preserving original character offsets for ground truth alignment. Short blocks of fewer than 50 tokens were merged into the next block if the combined length stayed under 512 tokens. Blocks exceeding 512 tokens were split at sentence boundaries.

Step 3 — Query Extraction: Each plagiarism annotation in the XML ground truth file was extracted as one query chunk, representing the plagiarized span in the suspicious document.

Step 4 — Ground Truth Construction (Qrels): For each query chunk, all source chunks whose character range overlapped with the corresponding source annotation span were marked as relevant. A single annotation span could overlap with more than one source chunk, making all overlapping chunks relevant for that query.

Chapter 7: Practical Implementation
